TIMING ANALYSIS AND OPTIMIZATION OF SEQUENTIAL CIRCUITS

TIMING ANALYSIS AND OPTIMIZATION OF SEQUENTIAL CIRCUITS

Naresh Maheshwari
Iowa State University

Sachin S. Sapatnekar
University of Minnesota



SPRINGER SCIENCE+BUSINESS MEDIA, LLC

Library of Congress Cataloging-in-Publication Data

Maheshwari, Naresh, 1970-
Timing analysis and optimization of sequential circuits / Naresh Maheshwari, Sachin S. Sapatnekar.
p. cm.
Includes bibliographical references and index.
ISBN 978-1-4613-7579-1 ISBN 978-1-4615-5637-4 (eBook)
DOI 10.1007/978-1-4615-5637-4
1. Integrated circuits--Very large scale integration--Design and construction--Data processing. 2. Computer-aided design. 3. Time-series analysis--Data processing.
I. Sapatnekar, Sachin S., 1967- . II. Title.
TK7874.75.M35 1998
621.39'5--dc21 98-42471
CIP

Copyright © 1999 by Springer Science+Business Media New York
Second printing 2001.
Originally published by Kluwer Academic Publishers in 1999
Softcover reprint of the hardcover 1st edition 1999
All rights reserved. No part of this publication may be reproduced, stored in a retrieval system or transmitted in any form or by any means, mechanical, photo-copying, recording, or otherwise, without the prior written permission of the publisher, Springer Science + Business Media, LLC.

Printed on acid-free paper.

Contents

List of Figures ix
Preface xiii

1. INTRODUCTION 1
   1.1 Performance Optimization of VLSI Circuits 1
   1.2 Outline of the Book 5

2. TIMING ANALYSIS OF SEQUENTIAL CIRCUITS 7
   2.1 Introduction 7
   2.2 Combinational Delay Modeling 10
   2.3 Clocking Disciplines: Edge-Triggered Circuits 17
   2.4 Resolving Short Path Violations 18
   2.5 Clocking Disciplines: Level-clocked Circuits 20
   2.6 Clock Schedule Optimization for Level-Clocked Circuits 26
   2.7 Timing Analysis of Domino Logic 27
   2.8 Concluding Remarks 31

3. CLOCK SKEW OPTIMIZATION 33
   3.1 The Notion of Deliberate Clock Skew 33
   3.2 Is Clock Skew Optimization Safe? 35
   3.3 Clock Tree Construction 35
   3.4 Clock Skew Optimization 40
   3.5 Clock Skew Optimization with Transistor Sizing 46
   3.6 Wave Pipelining Issues 60
   3.7 Deliberate Skews for Peak Current Reduction 62
   3.8 Conclusion 63

4. THE BASICS OF RETIMING 65
   4.1 Introduction to Retiming 65
   4.2 A Broad Overview of Research on Retiming 69
   4.3 Modeling and Assumptions for Retiming 72
   4.4 Minperiod Optimization of Edge-triggered Circuits 74

4.5 Level-clocked Circuits 88
4.6 Concluding Remarks 97

5. MINAREA RETIMING 99
5.1 The Leiserson-Saxe Approach 100
5.2 The Minaret Algorithm 105
5.3 Minarea Retiming of Level-Clocked Circuits 113

6. RETIMING CONTROL LOGIC 123
6.1 Minperiod Initial State Retiming via the State Transition Graph 124
6.2 Minperiod Initial State Retiming via Reverse Retiming 127
6.3 Minarea Initial State Retiming 129
6.4 Maintaining Initial States With Explicit Reset Circuitry 144

7. MISCELLANEOUS ISSUES IN RETIMING 147
7.1 Retiming and Testing 148
7.2 Verification Issues 154
7.3 Retiming for Low Power 156
7.4 Retiming with Logic Synthesis 157
7.5 Retiming for FPGA's 161
7.6 Practical Issues 162
7.7 Conclusion 168

8. CONCLUSION 169

References 171

2 TIMING ANALYSIS OF SEQUENTIAL CIRCUITS

2.1 INTRODUCTION

A general sequential circuit is a network of computational nodes (gates) and memory elements (registers). The computational nodes may be conceptualized as being clustered together in an acyclic network of gates that forms a combinational logic circuit. A cyclic path in the direction of signal propagation is permitted in the sequential circuit only if it contains at least one register. In general, it is possible to represent any sequential circuit in terms of the schematic shown in Figure 2.1, which has I inputs, O outputs and M registers. The registers outputs feed into the combinational logic which, in turn, feeds the register inputs. Thus, the combinational logic has $I+M$ inputs and $O+M$ outputs.

Pipelined systems, such as the one shown in Figure 2.2, are a special subset of the class of general sequential circuits, and are primarily used in datapaths. A pipelined system uses registers to capture the output of each logic stage at the end of each clock period. Data proceeds through each combinational block, or pipeline stage, until it reaches the end of the pipeline, with the registers serving to isolate individual stages so that they may parallelly process data corresponding to different data sets. The use of this parallelism causes an increase in the data throughput rate of the system. While many pipelines have no feedback, the definition of a pipeline does not preclude the use feedback, and algorithms for scheduling data into a pipeline with feedback have been

N. Maheshwari et al., *Timing Analysis and Optimization of Sequential Circuits*
© Kluwer Academic Publishers 1999

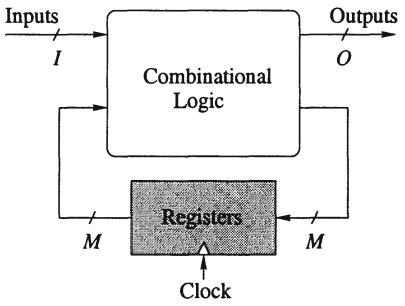


Figure 2.1. A general sequential circuit.

devised [HB84]. Acyclic pipelines form an important subclass of circuits and are popularly used in data-oriented designs as they are much simpler to design, analyze and optimize than general sequential circuits.

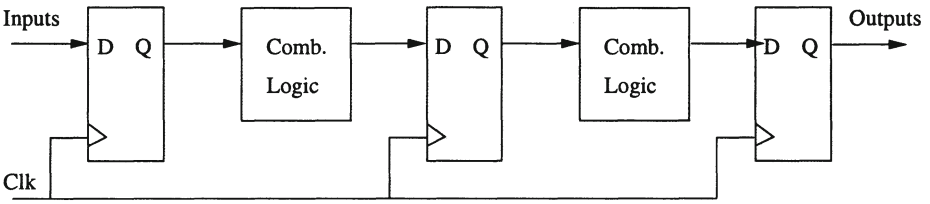


Figure 2.2. An example of a pipelined system.

The functioning of registers is key to the behavior of any sequential circuit, and there are several choices of register type that are available to a designer. The behavior of each register is controlled by the clock signal, and depending on the state of the clock, the data at the register input is either effectively isolated from its output, or transmitted to the output. Several types of registers may be used in a synchronous system, and are differentiated by the manner in which they use the clock to transmit data from the input to the output.

Level-clocked latches These are commonly referred to as latches, and permit data to be transmitted from the input to the output whenever the clock is high.

Edge-triggered flip-flops These are commonly called flip-flops (FF's), and use the clock edge to determine when the data is transmitted to the output. In a positive [negative] edge-triggered FF, data is transmitted from the input to the output when the clock makes a transition from 0 to 1 [1 to 0]. FF's can typically be constructed by connecting two level-clocked latches in a master-slave fashion; for details, see [WE93].

A circuit in which all memory elements are level-clocked latches is commonly referred to as a level-clocked circuit, and a circuit composed of edge-triggered FF's is called an edge-triggered circuit.

Other types of storage elements include the RS latch, the JK flip-flop, the T flip-flop, etc. We will not address these in our discussion and will primarily restrict ourselves to level-clocked and edge-triggered registers described above. A description of the transistor-level details of these registers is beyond the scope of this book, but we point out that various choices are available, and they influence the number of transistors used, the number of clock signals to be routed, and the area of and power dissipated by the chip.

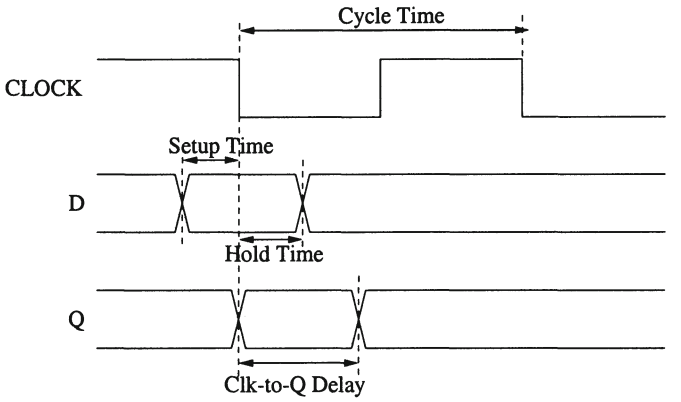


Figure 2.3. Illustration of the clocking parameters for a register.

The data input of the register, commonly referred to as the D input, must receive incoming data at a time that is at least T_{setup} units before the onset of the latching edge of the clock. The data will then be available at the output node, Q, after the latching edge. The input, D, must be kept stable for a time of T_{hold} units so that the data is allowed to be stored correctly. The quantities, T_{setup} and T_{hold}, are, unsurprisingly, referred to as the setup and hold times of the latch, respectively. Additionally, each latch has a delay between the time the data and clock are both available at the input, and the time when it is latched; this is referred to as the clock-to-Q delay, T_q.

For a negative edge-triggered register, the setup and hold time requirements are illustrated in Figure 2.3. The clock is a periodic waveform that repeats after every P units of time, where P is called the clock period or the cycle time.

The organization of this chapter is as follows. We will first survey, in brief, techniques for combinational delay modeling. Next, we will consider issues related to correct clocking, first for edge-triggered circuits and then for level-clocked circuits. During this discussion, we will describe techniques for setting up a description of the timing constraints for a circuit, and while this chapter is aimed mainly at timing analysis, we will periodically touch briefly on issues in timing optimization as well.

2.2 COMBINATIONAL DELAY MODELING

Finding the delay of a digital circuit accurately is an important part of the design and verification process. Various levels of simulation can be used, depending on the accuracy desired and the amount of CPU time that is affordable.

Of all techniques currently used to simulate a circuit, circuit-level simulation [CL75] provides the highest degree of accuracy. This method attempts to solve the circuit equations, i.e., the Kirchoff's current and voltage law equations, in conjunction with the device characteristics to find the voltage at every node and the current through every branch of the circuit. The process involves solving systems of nonlinear differential equations. The chief drawbacks of this method are the high computational cost and extensive memory requirements. In addition, due to various problems associated with nonlinear algebraic and differential equations, the convergence of the solution process is not always guaranteed.

To circumvent these disadvantages, techniques such as timing simulation, logic simulation and mixed-mode simulation may be used to compute waveforms of interest using various approximations, to achieve improved computational times at the expense of, it is hoped, small, sacrifices in accuracy.

2.2.1 Gate Delay Models

Often, a rough estimate of the circuit delay can suffice for solving macroscopic design issues, and one need not pay attention to precise voltage or current waveforms. In this context, it is useful to define an important parameter of a voltage waveform, namely, the delay associated with a transition. The transition delay is often defined as the amount of time required by the output waveform to cross the 50% threshold, after the input waveform has crossed its 50% threshold. Several other definitions are possible, but this is arguably the most commonly used one.

Since CMOS logic offers a good deal of modularity in that the analysis of the waveform of a gate can frequently be decoupled from the analysis of the waveform of most other gates in the circuit, modeling at the gate level is an important piece of the combinational timing analysis puzzle. Several approaches have been proposed for delay estimation in a computationally cheap manner. These approaches usually fall in one of the following classes:

- Modeling using RC equivalents: The gate is replaced by an equivalent RC circuit, and the delay of the RC circuit is computed. The advantage of this approach is that the RC circuit delay is typically much easier to estimate than the delay of the gate, and this leads to a significant computational advantage, with some loss in accuracy.

- Macromodeling of template gate types, such as inverters, NANDs, NORs, XORs and complex gates such as AOI and OAI gates, or certain composite cells, such as that in Figure 2.4, by deriving formulæ for the transition time.

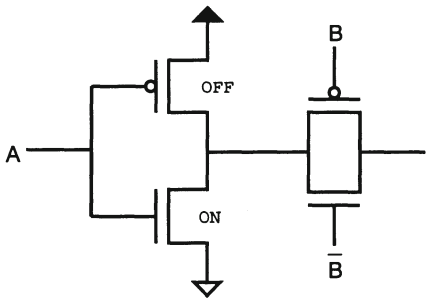


Figure 2.4. An example of a composite cell.

The use of resistors and capacitors for transistor level modeling has several advantages over more detailed simulation procedures, although it involves a loss in accuracy. In addition to the major reduction in the computational complexity, the RC representation often offers a simple closed-form formula for the delay of a circuit. Such simple formulæ are particularly useful for delay optimization, since it is far easier, in terms of tractability and computational cost, to perform mathematical operations on this relatively simple delay expression. Another benefit provided by this technique is that the effects of interconnect delay can easily be incorporated by augmenting the RC network that describes the MOS transistor netlist, with additional resistors and capacitors that represent interconnect lines.

The Elmore delay is a good approximation to the dominant time constant of the step response [Wya87]. This is adequate in situations where one time constant dominates; however, for ultra-deep submicron circuits, it is likely that more complex techniques may be required. Reduced order modeling techniques, such as asymptotic waveform evaluation (AWE) [PR90], Padé via Lanczos (PVL) [FF95] and PRIMA [OP97], extend the Elmore idea to nondominant, but important, time constants of the circuit, and may be used.

To properly consider the effects of driver nonlinearities, it is appropriate to consider timing models that derive explicit closed-form solutions for the situation where a driver drives a load that may be either purely capacitive, or an RC load modeled directly or as an effective capacitance [QPP94], or an RLC line. The basic unit in this approach is the channel-connected component (henceforth referred to simply as a component). Each component corresponds to a set of transistors that are connected by drain and source nodes, and the delay model captures the effect of the component driving an interconnect line. A detailed explanation of these approaches is beyond the scope of this book, but the reader is referred to, for example, [SK93, CN94, DMQP94].

Elmore Delay Calculations. The Elmore delay is a useful way of estimating the dominant time constant of an RC circuit. For a fairly complete discussion of Elmore delays, the reader is referred to [SK93]; we will overview the basics of Elmore delay calculations for simple structures in this section.

A commonly-encountered structure is the RC tree, which is a network of resistors and capacitors such that there are no resistor loops (i.e., all of the resistors form a topological tree that does not include the ground node[1]), and all of the capacitors in an RC tree are connected to ground. One of the nonground nodes of the tree is assumed to be the input, and one or more nodes the outputs. Due to the tree structure, there is a unique path through the resistive path of the network from any nonground node to the input node. An example of an RC tree is shown in Figure 2.5. The input node of the network is the one marked *"IN"*.

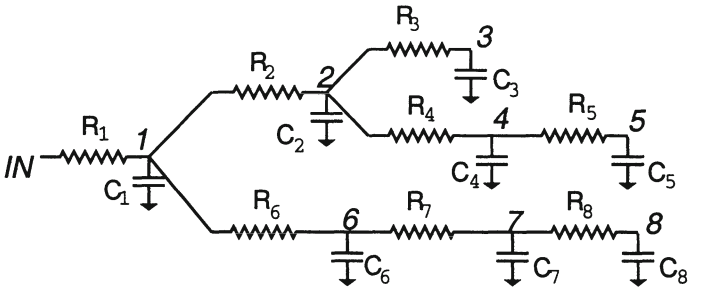


Figure 2.5. An example of an RC tree.

For an RC tree, the Elmore delay can be calculated by inspection as follows:

- Let P_i be the unique path from the input node $\mathbf{n}_0$ to node $\mathbf{n}_i$.
- Let $P_{ij} = P_i \cap P_j$ denote the portion of the path between $\mathbf{n}_0$ and $\mathbf{n}_i$ that is common to the path between $\mathbf{n}_0$ and $\mathbf{n}_j$.
- Let R_{ij} denote the sum of the resistances in P_{ij}; if $P_{ij} = \emptyset$, then $R_{ij} = 0$.

The Elmore delay to node i in the RC tree is then given by the expression

$$T_{Di} = \sum_{j=0}^{n} R_{ij} C_j. \tag{2.1}$$

Example : Consider the circuit shown in Figure 2.5. The node $\mathbf{n}_0$ is the input node. The procedure described above may be used to compute the Elmore delay to various nodes in the circuit. For example,

$$\begin{aligned} T_{D\mathbf{n}_7} &= R_1C_1 + R_1C_2 + R_1C_3 + R_1C_4 + R_1C_5 + (R_1 + R_6)C_6 \\ &\quad +(R_1 + R_6 + R_7)C_7 + (R_1 + R_6 + R_7)C_8. \end{aligned} \tag{2.2}$$

$$\begin{aligned} T_{D\mathbf{n}_5} &= R_1C_1 + (R_1 + R_2)C_2 + (R_1 + R_2)C_3 \\ &\quad +(R_1 + R_2 + R_4)C_4 + (R_1 + R_2 + R_4 + R_5)C_5 \\ &\quad +R_1C_6 + R_1C_7 + R_1C_8. \end{aligned} \tag{2.3}$$

This calculation is performed by conducting a postorder tree traversal [CLR90] to calculate the downstream capacitance for each node, followed by a preorder traversal to calculate the Elmore delay to each node.

2.2.2 *Delay Calculation at the Circuit Level*

At the circuit level, the delay calculation deals primarily with purely combinational logic circuits. An edge-triggered circuit can be decomposed into combinational logic blocks that lie between latches. Since each of these logic blocks has to satisfy a particular timing requirement dictated by system clock signals, it is the delay of a combinational logic block that has practical significance; the precise forms of these constraints are presented in Section 2.3.

The method that is commonly referred to as PERT (Program Evaluation and Review Technique) and is popularly used in timing analysis is actually a misnomer; it refers to the CPM (Critical Path Method) in project management. In reality, PERT is a technique that is used to propagate probabilistic delay information through a network, while CPM propagates static delay information. The confusion probably arose because one of the earliest works on timing analysis by Kirkpatrick and Clark [KC66] used PERT for statistical timing analysis[2]. These principles were extended by later work for scalar delay values, without necessarily changing the nomenclature. For an overview of the genesis of timing analysis methods, the reader is referred to an early paper by Hitchcock [Hit82]. The CPM procedure is now an integral part of most fast algorithms for circuit delay calculation.

The procedure is best illustrated by means of a simple example. Consider the circuit in Figure 2.6. Each box represents a component. The number within the box represents the delay associated with it. We assume that the worst-case arrival time for a transition at any primary input, i.e., at the inputs to boxes A, B, C, and D, is 0.

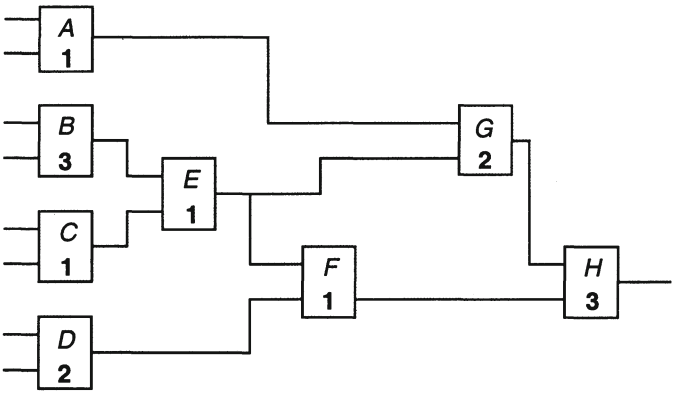


Figure 2.6. The Critical Path Method (CPM).

A component is said to be *ready for processing* when the signal arrival time information is available for all of its inputs. Initially, since signal arrival times are known only at the primary inputs, only those components that are fed solely by primary inputs are ready for processing. In the example, these components would be A, B, C, and D. These are placed in a queue and are processed in the order in which they appear in the queue.

In the iterative process, the component at the head of the queue is scheduled for processing. Each processing step consists of

- Finding the latest arriving input to the component, which triggers the output transition. This involves finding the maximum of all worst-case arrival times of inputs to the component.
- Adding the delay of the component to the latest arriving input time, to obtain the worst-case transition time at the output.
- Checking all of the components that the current component fans out to, to find out whether they are ready for processing. If so, the component is added to the tail of the queue.

The iterations end when the queue is empty.

In the example, the algorithm is executed as follows:

Step 1. In the initial step, A, B, C, and D are placed in the queue.

Step 2. A is scheduled. The latest input transition for A is 0, the delay of A is 1; hence, the latest output transition for A is at time (0+1) = 1. No additional elements can be added to the queue.

Step 3. B is scheduled. The latest output transition is found to be at time (0+3) = 3. No additional elements can be placed in the queue.

Step 4. C is scheduled, and the worst-case output transition occurs at time (0+1) = 1. At this point, all input information for component E is available, and it is placed at the tail of the queue.

Step 5. D is scheduled; the output transition time is found to be (0+2) = 2.

Step 6. E is scheduled; the latest arriving input comes in at time 3; hence, the worst-case output transition occurs at time (3+1) = 4. At this point, F and G are added to the tail of the queue.

Step 7. F is scheduled; its latest output transition is at time (4+1) = 5.

Step 8. G is scheduled; its latest output transition occurs at time (4+2) = 6. H is added to the tail of the queue.

Step 9. H is scheduled; its worst-case output transition is at time (6+3) = 9. The queue is now empty and the algorithm terminates.

The worst-case delays at the output of each component are shown in Figure 2.7.

The *critical path*, defined as the path between an input and an output with the maximum delay, can now easily be found by using a traceback method. We begin with the component whose output is the primary output with the latest transition time. This is the last component on the critical path. Next, the latest arriving input to this component is identified. The component that causes this transition is the preceding component on the critical path. The process is repeated recursively until a primary input is reached.

In the example, we begin with component H at the output. The latest arriving input to this component is caused by component G, which thus precedes H on the critical path. Similarly, the transition at G is caused by the latest arriving input, which comes in from component E, and so on. By continuing this process, the critical path from the input to the output is identified as B-E-G-H.

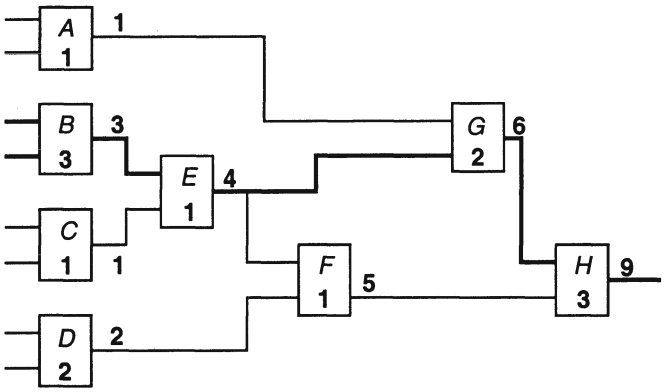


Figure 2.7. Circuit delays obtained from the Critical Path Method.

The method described above can easily be adapted to find the *minimum*, instead of the maximum delay from any input to any output. The only changes in the procedure involve the manner in which an individual component is processed: the earliest arrival time over all inputs is now added to the delay of the component to find the earliest arrival time at its output.

In the case of CMOS circuits, the rise and fall delay transitions are calculated separately. For inverting CMOS gates, the latest arriving input rise (fall) transition triggers off a fall (rise) transition at the output. This can easily be incorporated into CPM described above, by maintaining two numbers, t_r and t_f, for each gate, corresponding to the worst-case rise (high transition) and fall (low transition) delays from a primary input. To obtain the value of t_f at an output, the largest value of t_r at an input node is added to the worst-case fall transition time of the component; the computation of t_r at an output is analogous. For noninverting gates, t_r and t_f are obtained by adding the rise (fall) transition time to the worst-case input rise (fall) transition time.

The CPM procedure does not incorporate the role of false paths, i.e., paths that cannot be excited by any set of input signals. For a detailed treatment of false paths, the reader is referred to [MB91, YH97].

2.2.3 Calculating the Worst-case Delays from a Specific Input to All Outputs

The CPM procedure described above finds the worst-case delay from any input to any output in a combinational block. A commonly encountered problem (for example, see Section 3.4) is that of finding the maximum delay, $\overline{d}(i,j)$, from *one* specified input to all outputs. The work in [Fis90] suggested a procedure for computing $\overline{d}(i,j)$ for all j. The arrival time at input i is set to zero, and that at all other inputs to $-\infty$. The resulting signal arrival time at each output j, found using CPM, is the value of $\overline{d}(i,j)$. However, if this procedure were to be performed directly, it would lead to large computation times. This would be accentuated if all input-to-output delays were required, which would mean that

this procedure would have to be executed for each input to the combinational block.

For practical circuits, it was observed during the symbolic propagation of constraints in [CSH95] (described in Section 3.5.1) that in most cases, the input to a combinational block exercises only a small fraction of all of the paths between the inputs of the combinational block and the outputs. Based on this observation, an efficient procedure for calculating the values of $\overline{d}(i,j)$ was developed in [Sap96]. It was found that the use of this procedure yielded run-time improvements of several orders of magnitude over the direct multiple applications of CPM described in the previous paragraph.

The level number, level(k), of each gate k in the circuit is first computed by a single CPM run; the level number is defined as the largest number of gates from a primary input to the gate, inclusive of the gate. In other words, the level number of a gate is found by a topological ordering algorithm. To find $\overline{d}(i,o)$, the largest delay from primary input i to all primary outputs o, an event-driven CPM-like exercise is conducted, starting at flip-flop i, as described in the following piece of pseudocode. During the process, a set of queues, known as level queues, is maintained. The level queue indexed by k contains all gates at level k that have had an input processed. The pseudocode for the procedure is as follows:

```
Algorithm ALL_OUTPUT_DELAYS_FROM_SINGLE_INPUT
Initialize all level queues to be empty;
currentlevel = 0;
currentgate = input i;
while (currentgate ≠ nil) {
   if (currentgate ≠ i)
      for (all fanins j of currentgate)
         d̄(i,currentgate) = max(d̄(i,j)) + delay(currentgate);
   else
         d̄(i,i) = 0;
   for (all fanouts k of currentgate)
      Append gate k to the level queue indexed by level(k),
         if it does not already lie on the queue;
   if (all level queues are not empty) {
      currentlevel = min.  level number with nonempty level queue;
      currentgate = head of the level queue for currentlevel;
      delete currentgate from its level queue;
   }
   else
      currentgate = nil;
}
```

At each step, an element from the lowest unprocessed level is plucked from its level queue, and the worst-case delay from flip-flop i to its output is computed. All of its fanouts are then placed on their corresponding level queues, unless

they have already been placed on these queues. Note that by construction, no gate is processed until the delay to all of its inputs that are affected by flip-flop i have been computed, since such inputs must necessarily have a lower level number.

2.3 CLOCKING DISCIPLINES: EDGE-TRIGGERED CIRCUITS

So far in this chapter, we have focused primarily on timing analysis techniques for combinational circuits. While this is an important problem, the real utility of these procedures is in considering a combinational circuit in the context of how it is embedded in a sequential circuit, as shown in Figure 2.1, and determining whether the sequential circuit meets the requirements imposed by the clock signal or not. We will first perform this analysis for edge-triggered circuits.

In the edge-triggered scenario, let us consider two FF's, i and j, connected only by purely combinational paths. Given all such paths $i \rightsquigarrow j$, let the upper bound on the delay $d(i,j)$ from FF i to FF j be $\overline{d}(i,j)$, and the lower bound be $\underline{d}(i,j)$, i.e.,

$$\underline{d}(i,j) \leq d(i,j) \leq \overline{d}(i,j)$$

We will denote the setup time, hold time and clock-to-Q delay of any arbitrary FF k as T_{s_k}, T_{h_k}, and T_{q_k} respectively, and the clock period as P.

The data is available at the launching FF, i, at time T_{q_i}, and will arrive at the latching FF, j, at a time no later than $T_{q_i} + \overline{d}(i,j)$. For correct clocking, the data is required arrive one setup time before the latching edge of the clock at FF j, i.e., at a time no later than $P - T_{s_j}$. This leads to the following constraint:

$$\begin{aligned} T_{q_i} + \overline{d}(i,j) &\leq P - T_{s_j} \\ \text{i.e., } \overline{d}(i,j) &\leq P - T_{s_j} - T_{q_i} \end{aligned} \tag{2.4}$$

For obvious reasons, this constraint is often referred to as the *setup time constraint.* Since this requirement places an upper bound on the delay of a combinational path, it is also called the *long path constraint.* A third name attributable to this is the *zero clocking constraint,* so christened because the data will not arrive in time to be latched at the next clock period if the combinational delay does not satisfy this constraint.

The data must be stable for an interval that is at least as long as the hold time after the clock edge, if it is to be correctly captured by the FF. Hence, it is essential to ensure that the new data does not arrive at FF j before time T_{h_j}. Since the earliest time that the incoming data can arrive is $T_{q_i} + \underline{d}(i,j)$, this gives us the following *hold time constraint*:

$$\begin{aligned} T_{q_i} + \underline{d}(i,j) &\geq T_{h_j} \\ \text{i.e., } \underline{d}(i,j) &\geq T_{h_j} - T_{q_i} \end{aligned} \tag{2.5}$$

Since this constraint places a lower bound on the combinational delay on a path, it is referred to as a *short path constraint.* If the constraint is violated, then the data in the current clock cycle is corrupted by the data from the next clock cycle; as a result, data is latched twice instead of once in a clock cycle. Hence, this constraint is also called the *double clocking constraint.* Notice that if the clock-to-Q delay of FF i is greater than the hold time of FF j, i.e., $T_{q_i} \geq T_{h_j}$ (this condition is normally not true in practice), then the right hand side of the constraint is negative. In this case, since $\underline{d}(i,j) \geq 0$, the short path constraint is always satisfied.

2.4 RESOLVING SHORT PATH VIOLATIONS

Correct circuit operation requires that all data paths have a delay that lies between an upper bound and a lower bound to satisfy long path and short path constraints. Traditional approaches in delay optimization for combinational circuits have dealt with methods to decrease the delay of the longest path, frequently neglecting to consider the problem of short path violations that may already exist, or may be introduced as a result of the optimizations.

In [SBS93], the issue of satisfying the lower bound delay constraints for edge-triggered circuits is addressed. The short path constraints are handled as a post-processing step after traditional delay optimization techniques by padding delays into the circuit to ensure that it meets all short path constraints. The problem of inserting a minimum number of such delays is referred to as the minimum padding problem. Two methods for solving the problem are described in [SBS93]:

(1) The first simple algorithm finds an edge on a short path that does not lie on any critical long path. A delay is inserted on the edge until one of the following two cases becomes true:

   (a) either some path containing the edge becomes a critical long path, in which case no further delays are added on this edge, and the process is continued for the edge that is identified next, or

   (b) the short path containing the edge meets the lower bound and thus success is achieved in padding.

   This procedure is carried for all the short paths until the lower bound delay constraints are met. This procedure is purely a heuristic and has no guarantees of optimality.

(2) The second approach formulates the problem as a linear program. The combinational circuit is represented as a directed acyclic graph with every gate output, primary input or primary output in the circuit represented by a vertex in the graph. Two type of edges are defined, namely, internal and external edges, denoted by E^I and E^E. An internal edge is directed from an input pin of a gate to its output pin. The external edges represent the interconnections or wiring in the circuit. Hence, if an edge $e_{i,j} \in E^I$, then

the weight $w_{i,j}$ of the edge is the delay of the gate from the input represented by i to the output of the gate. If the edge $e_{i,j} \in E^E$, then the weight $w_{i,j}$ of the edge is a variable whose value is to be determined; the physical meaning of the variable $w_{i,j}$ here is the number of delays padded on the edge $e_{i,j}$. Delays may be inserted only on the external edges. There are two arrival times associated with each vertex i: an early arrival time a_i and a late arrival time A_i. If i is a primary input (PI), then a_i and A_i are specified by the user or the algorithm at the higher level, say, as $a_i = \lambda_i, A_i = \Lambda_i$. If i is an input/output pin of a gate, then

$$\begin{aligned} a_i &= \min_{j \in FI(i)} (a_j + w_{j,i}) \\ A_i &= \max_{j \in FI(i)} (A_j + w_{j,i}) \end{aligned} \tag{2.6}$$

where $FI(i)$ denotes the set of fanins of i. A path is a sequence of vertices, and the delay of a path p is denoted by $d(p)$. At every primary output (PO) i, the data is required to be available no earlier than r_i and no later than R_i; namely, $r_i \leq a_i \leq A_i \leq R_i$. A path p fails to meet the short path constraint if $a_i + d(p) \leq r_i$. In that case, delays $w_{i,j}$ are added along the edge (i,j) on the short path to remove this violation. The area overhead of this addition is proportional to the number of delays added, and is approximated as $\sum_{e_{i,j} \in E^E} w_{i,j}$. The problem of finding the minimum-cost padding that satisfies the timing constraints can now be expressed as a linear program, with constraints obtained by relaxing equation (2.6), and an objective function that corresponds to the number of buffers added.

$$\begin{aligned} \text{minimize} \quad & \sum_{e_{i,j} \in E^E} w_{i,j} \\ \text{subject to} \quad & A_j \geq A_i + w_{i,j} \quad \forall i \in FI(j) \\ & a_j \leq a_i + w_{i,j} \quad \forall i \in FI(j) \\ & A_i \leq R_i \quad \forall i \in PO \\ & a_i \geq r_i \quad \forall i \in PO \\ & A_i = \Lambda_i \quad \forall i \in PI \\ & a_i = \lambda_i \quad \forall i \in PI \end{aligned} \tag{2.7}$$

It was shown that in any circuit, if for every pair $i_1 \in PI$ and $i_n \in PO$ such that $\exists$ a combinational path $p : i_1 \rightsquigarrow i_n$, we are given that $A_{i_1} - a_{i_1} \leq R_{i_n} - r_{i_n}$, then the padding problem can be solved. An intuitive explanation for this condition is as follows; if we interpret the term, $A_{i_1} - a_{i_1}$, as the uncertainty interval in the arrival time of the signal at the primary input, i_1, and the term, $R_{i_n} - r_{i_n}$, as the uncertainty in the required time at the output, then since the circuit is causal, it cannot make the uncertainty interval at the output any narrower than the uncertainty interval at its input.

2.5 CLOCKING DISCIPLINES: LEVEL-CLOCKED CIRCUITS

2.5.1 Introduction

As mentioned in Section 2.1, a latch, unlike an FF, is transparent during the active period of the clock. This makes the analysis and design of level-clocked circuits more complex than edge-triggered circuits, since combinational blocks are not insulated from each other by the memory elements, and multicycle paths are possible when the data is latched during the transparent phase of the clock. Even though this transparent nature introduces an additional level of complexity to the design phase, level-clocked circuits are widely used for high-performance designs since they offer more flexibility than edge-triggered circuits, both in terms of the minimum clock period achievable and the minimum number of memory elements required.

As an illustration of this notion, consider the simple circuit in Figure 2.8 with unit delay gates and a single-phase clocking scheme with a 50% duty cycle. Let us assume that the data signals are available at the primary inputs at the falling edge of the clock, and must arrive at the primary outputs before the falling edge. At the level-triggered latch L1, the data may depart at any time while the clock is high. A data signal in this circuit is allowed precisely two clock periods to reach the primary output from the primary input.

Consider the operation of the circuit under a clock period of 2 units; notice that the path delay between latch L1 and the output is more than the clock period. However, the circuit works correctly due to the transparent nature of the latches. As shown in the figure, the data departs from the IN node at time 0, arrives at and departs from the latch L1 at time 1, and is latched at the output at time 4, which corresponds to the onset of the second clock edge. In contrast, if L1 were an edge-triggered FF, then a clock period of 2 units would have been untenable, and the minimum possible clock period would have been 3 units. This example demonstrates the advantages of using level clocked circuits. This practice of using the active period of the clock to allow combinational paths to have a delay of more than the clock period is referred to variously as either cycle borrowing, cycle stealing, slack stealing, or time borrowing.

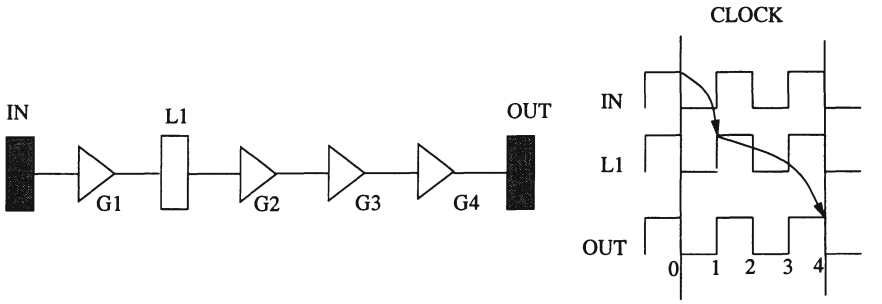


Figure 2.8. Example illustrating the advantages of using level-sensitive latches.

For edge-triggered circuits, the insulating nature of the memory elements leads to the requirement that the delays through all combinational logic paths must be less than the clock period, with an allowance for setup and hold time constraints. Therefore, timing constraints need only be enforced between FF's connected by a purely combinational path. For level-clocked circuits, the delay through a combinational logic path may be longer than one clock cycle, as long as it is compensated by shorter path delays in subsequent cycles.

To ensure that the extra delay is compensated, we must enforce timing constraints between a latch and every other latch reachable from it via a path that traverses combinational logic, and possibly multiple latches.

Example. Consider a linear N stage pipeline with $N+1$ memory elements $(m_0, m_1 \ldots m_N)$. If these memory elements were edge-triggered FF's, then we would need only N timing constraints $(m_i \rightsquigarrow m_{i+1},\ 0 \leq i \leq N)$. However, if these memory elements were to be level sensitive latches, then we would need $N \cdot (N+1)/2$ timing constraints $(m_i \rightsquigarrow m_j\ \forall j > i$ and $0 \leq i \leq N)$ to check the correctness of multicycle paths. In the presence of feedback paths, the timing analysis of level-clocked circuits would become even more complex.

As will soon be shown, some of this complexity can be reduced by the introduction of appropriate intermediate variables.

2.5.2 Clock Model

Single-phase and Multiphase Clocking. The most conventional form of clocking uses a single-phase clock, where every rising [falling] clock edge is perfectly aligned with every other rising [falling] clock edge, assuming that no skews are introduced by the clock distribution network. High-performance circuits may, instead, use multiphase clocking; in a k-phase clock, the k phases typically have the same clock period, but are staggered from each other by a fixed time delay. Each phase consists of two intervals: an active interval during which the latches are enabled, and a passive interval when they are disabled; these typically correspond to the interval when the clock signal is high and low, respectively.

Depending on the way the clock scheme is designed, the phases may be overlapping or nonoverlapping, i.e., their active times may or may not simultaneously intersect in time with any other phase. Clocking schemes may be designed to be symmetric, which implies that all phases have an equal active time, and the time difference between the rising clock edges for successive phases i and j are all equal; a clock scheme that does not satisfy this property is said to be asymmetric.

Although it is not essential for correct operation, if one were to traverse any path in the circuit in the direction of signal propagation, one would typically find that a latch of phase i would be followed by a latch of phase $(i+1) \bmod k$; if so, the circuit is referred to as a well-formed circuit.

The SMO Clock Model. The clock model used by Sakallah, Mudge and Olukotun [SMO92], so named after the initials of its authors, presents a complete description of a multiphase clock signal; this model will be used later in this chapter to describe the set of timing constraints for a level-clocked circuit, as derived by their work.

A k-phase clock is a set of k periodic signals, $\Phi = \{\phi_1 \ldots \phi_k\}$, where $\phi_i, 1 \leq i \leq k$ is referred to as phase i of the clock Φ. All of the ϕ_i's have the same clock period, P. Each phase, i, has an active interval of duration T_{ϕ_i}, and a passive interval of duration $(P - T_{\phi_i})$. Let Ψ denote the set of all latches. Each latch, $j \in \Psi$, is clocked by exactly one phase of the clock Φ, which is denoted by $p(j)$. All of the latches controlled by a clock phase are enabled during the active interval and disabled during the passive interval. Without loss of generality, it will be assumed in this discussion that the active interval corresponds to the clock being at logic value 1, while in the passive interval, the clock is at logic 0. The term "clocking scheme" is used to indicate the relative locations of the clock transition edges, and the duty cycles of the individual phases. Thus, a clocking scheme, together with a clock period, P, defines a clock schedule, Φ.

Associated with each phase, i, is a *local time zone*, shown in Figure 2.9, such that the passive interval starts at time 0, the enabling edge occurs at time $(P - T_{\phi_i})$, and the latching edge occurs at time P. Phases are ordered so that $e_1 \leq e_2 \ldots \leq e_{k-1} \leq e_k = P$, and are numbered modulo-k, i.e., $\phi_{k+1} = \phi_1$ and $\phi_{1-1} = \phi_k$. A global time reference is maintained as a general reference, and e_i denotes the time when the phase ϕ_i ends, relative to this global reference.

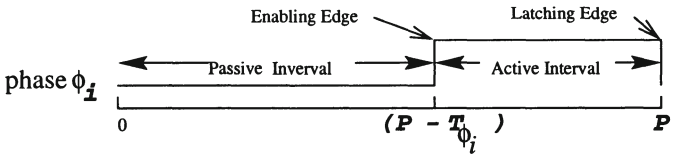


Figure 2.9. Phase i of a k-phase clock with all times relative to the local time zone.

The phase shift operator, $E_{i,j}$, illustrated with the help of Figure 2.10, is defined as follows:

$$E_{i,j} = \begin{cases} (e_j - e_i) & \text{for } i < j \\ (P + e_j - e_i) & \text{for } i \geq j \end{cases} \tag{2.8}$$

Note that $E_{i,j}$ always takes on positive values, and acts as a readjustment factor in going from one time zone to another. Specifically, when $E_{i,j}$ is subtracted from a time point in the current time zone of ϕ_i, it changes the frame of reference to the local time zone of ϕ_j.

The clock is required to satisfy the following constraints:

$$T_{\phi_i} \leq P \tag{2.9}$$

$$0 \leq e_i \leq P \tag{2.10}$$

$$e_i \leq e_{i+1} \tag{2.11}$$

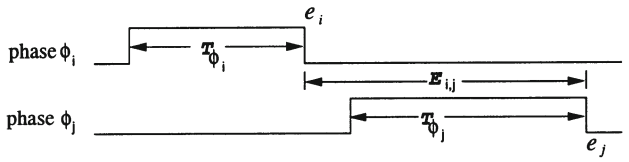


Figure 2.10. Illustration of the phase shift operator.

$$P \geq 0 \tag{2.12}$$

$$T_{\phi_i} \geq 0 \tag{2.13}$$

$$E_{i,j} \geq T_{\phi_j} \tag{2.14}$$

The constraints (2.9), (2.10), (2.12) and (2.13) follow from the definition of the clock. The constraint (2.11) maintains an ordering of the phases, and (2.14) ensures that the phases are nonoverlapping. The last of these may be relaxed in case of an overlapping clock.

2.5.3 *Timing Constraints for Level-Clocked Circuits*

We now enumerate the set of timing constraints that dictate the correct operation of a level-clocked circuit. As before, the parameters $T_{s,i}$ and $T_{h,i}$ represent the setup and hold times, respectively, of the ith latch, and we assume δ_i and Δ_i to be the minimum and maximum delays through each latch. In consistency with previously introduced notation, for each pair of latches, i and j, connected by a combinational path $i \rightsquigarrow j$, the maximum and minimum path delays are denoted by $\overline{d}(i,j)$ and $\underline{d}(i,j)$, respectively.

The data input to each latch, i, has an associated arrival time that lies in the range between the earliest time, a_i, and the latest time, A_i. Similarly, the earliest and latest departure time of the data are denoted by d_i and D_i, respectively. All of these times are with respect to the local time zone of the latch.

Due to the transparent nature of the latches, a signal can depart from latch i at any time during the active interval of the phase $p(i)$, i.e.,

$$P - T_{\phi_{p(i)}} \leq d_i \leq D_i \leq P$$

However, a signal cannot depart from a latch before it has arrived at that latch, i.e.,

$$A_i \leq D_i$$
$$a_i \leq d_i$$

We can combine these constraints to obtain

$$D_i = \max\{A_i, P - T_{\phi_{p(i)}}\}$$
$$d_i = \max\{a_i, P - T_{\phi_{p(i)}}\}$$

The arrival time at a latch, j, of a signal departing from another latch, i, connected by one or more purely combinational paths, $i \rightsquigarrow j$, must satisfy the following relations

$$\begin{aligned} D_i + \Delta_i + \overline{d}(i,j) - E_{p(i),p(j)} &\leq A_j \\ d_i + \delta_i + \underline{d}(i,j) - E_{p(i),p(j)} &\geq a_j \end{aligned}$$

In addition, the latest arrival time must be at least one setup time before the falling edge of the clock, and the earliest arrival time must occur no sooner than a hold time after the rising edge of the clock.

Combining all of these relations, we can obtain the timing constraints that must be imposed for a level clocked circuit to be correctly clocked as

$$A_j = \max_{i \rightsquigarrow j; i,j \in \Psi} \{D_i + \Delta_i + \overline{d}(i,j) - E_{p(i),p(j)}\} \tag{2.15}$$

$$D_i = \max(A_i, P - T_{p(i)}) \; \forall \, i \in \Psi \tag{2.16}$$

$$A_i \leq P - T_{s,i} \; \forall \, i \in \Psi \tag{2.17}$$

$$a_j = \min_{i \rightsquigarrow j; i,j \in \Psi} \{d_i + \delta_i + \underline{d}(i,j) - E_{p(i),p(j)}\} \tag{2.18}$$

$$d_i = \max(a_i, P - T_{p(i)}) \; \forall \, i \in \Psi \tag{2.19}$$

$$a_i \geq T_{h,i} \; \forall \, i \in \Psi \tag{2.20}$$

A critical path of a circuit corresponds to the path corresponding to a sequence of constraints associated with the gates and latches on the path, such that each of these constraints is an active or violated constraint. A critical path in a level-clocked circuit would typically traverse several combinational and sequential circuit elements. The term "critical path of a circuit" is often used colloquially to refer to the path with the largest violation.

Critical paths may be classified into three categories:

Critical long paths are associated with long path constraints, or setup time constraints. Any increase in the delay along the path will worsen the slack or violation along that path.

Critical short paths are those that are associated with short path or hold time constraints. These paths are sensitive to a reduction in the delay along the path; any such reduction is liable to worsen the slack or violation along the path.

Critical cycles correspond to cyclic critical paths in the graph along which the constraints are active or violated.

For correct operation of a circuit, timing violations along all critical paths must be resolved.

Timing Verification. Several procedures for timing analysis have been proposed, based on the SMO constraints, including [SMO92, SS92, Szy92, LTW94, BSM95]. In this section, we provide a brief synopsis of this work; the reader

is referred to these papers for further details. The procedure *checkTc* [SMO90] proposed an iterative procedure for resolving the timing constraints. It was shown in [SS92] that the system of equations can be solved by relaxation in polynomial time in the number of latches, until a fixed point is obtained. It was also shown that the original system of equations in the SMO formulation (not the relaxed versions!) have a unique solution unless the circuit operates at the minimum clock period. The following algorithm, `TIMING_VERIFY`, shows an outline of the procedure:

```
Algorithm TIMING_VERIFY
/* initialize the latch arrival times */
for all latches i
   A[i] = a[i] = -∞;
/* Iterate the evaluation of the departure and arrival time */
/* equations until convergence or a maximum of |L| iterations, */
/* where |L| is the number of latches.  */
iter = 0;
repeat
   iter++;
   for i = 1 to |L| {
       Aprev[i] = A[i]; aprev[i] = a[i];
   }   /* Update the latch departure time based on the latch */
   /* arrival times computed in the previous iteration */
   for i = 1 to |L| {
       D[i] = max(APrev[i],P - T_p(i));
       d[i] = max(aPrev[i],P - T_p(i));
   }
   /* Update the latch arrival time based on the just-computed */
   /* latch departure times */
   for j = 1 to |L| {
       A[j] = max_i (D[i] + Δ[i] + d̄(i,j) - E(p(i),p(j));
       a[j] = min_i (d[i] + δ[i] + d̲(i,j) - E(p(i),p(j));
   }
until ((A[i] == APrev[i]) && (a[i] == aPrev[i])) || (iter > |L| - 1);

/* Check and record setup and hold time violations */
for i = 1 to |L| {
    SetupVio[i] = (A[i] > P - T_s,i);
    HoldVio[i] = (a[i] < T_h,i);
};
```

The procedure is essentially a variant of the Bellman-Ford algorithm [CLR90], which will be described in detail in Section 3.4.4. In case the constraint set has some slacks, as is almost always the case, a general algorithm that finds values of each a_i, A_i, d_i and D_i that satisfies the above constraints is not obliged to report times that are physically realizable. In contrast, this procedure finds the

earliest arrival and departure times for the signals, corresponding to a physically meaningful solution.

Several variants of the procedure may be used to speed up the computation. For instance, by clipping off the departure times to physically realizable values (for example, setting $D_i = \min(\max(A_i, P - T_{p(i)}), P)$, the information detected is more relevant. If the signal does not arrive in time for the closing edge of the clock, it is signaled as a timing violation. This procedure serves to localize the effect of timing violations, and such a diagnostic provides useful input to procedures that are used to remedy such violations.

An alternative approach to this method utilizes the concept of back-edges [LTW94]. For a directed acyclic graph processed in topological order, the Bellman-Ford procedure (or the algorithm `TIMING_VERIFY` above) concludes in one iteration. It can be shown that if the graph has b back edges, b iterations are necessary. While the problem of identifying a minimal number of back-edges (i.e., the minimum feedback vertex set problem) is NP-hard, in real circuits, it is found that the number of back edges is small enough that almost any set is good enough to provide a good improvement over a naive implementation of the relaxation procedure above.

2.6 CLOCK SCHEDULE OPTIMIZATION FOR LEVEL-CLOCKED CIRCUITS

The clock schedule optimization problem involves finding the minimum clock period, and the corresponding locations of the rising and falling edges of each phase, so that the clocking scheme meets all timing constraints. Due to the freedom allowed in the arrival times for level-clocked circuits, this optimization can make a substantial difference to the behavior of the circuit.

The *minTc* algorithm [SMO90] formulates the clock schedule optimization problem as a linear program. The problem is stated as follows:

$$\begin{array}{ll} \text{minimize} & P \\ \text{subject to} & \text{Clock constraints in relations (2.9) through (2.14)} \\ & \text{Latch constraints in relations (2.15) through (2.17)} \end{array} \tag{2.21}$$

After relaxing all max operators in relations (2.15) through (2.17) by replacing them by "$\geq$" inequalities, the problem maps on to a linear program whose minimum objective function value can be proven to be the same as that of the original problem. Notice that this formulation considers the long path constraints only. In case short path constraints are to be considered, it is not possible to relax them in the same way as long path constraints [BSM95].

The work in [Szy92] developed techniques to reduce the number of long path constraints in the linear program. A principal idea was the concept of *relevant paths*. Before defining this concept, we observe that if $r(p)$ denotes the number of latches on a path $p : e_i \rightsquigarrow e_j$, and $w(p)$ is the delay along the path (with allowances for setup times), then it must be true that

$$w(p) \leq r(p) \cdot P + E_{p(i),p(j)} \tag{2.22}$$

A path, p, is said to be relevant for period P if for any other path, p', with the same endpoints as p, $[w(p') - r(p') \cdot P] < [w(p) - r(p) \cdot P]$ if $r(p') < r(p)$, and $w(p') \leq w(p)$ if $r(p') = r(p)$. It was shown that in order to check the feasibility of a clock schedule, it suffices to check if the constraints on the set of relevant paths are satisfied; the principle behind this is that satisfying the relevant constraint will ensure that other constraints that were not relevant will be forced to satisfy relation (2.22).

The work in [Szy92] uses this pruning to reduce the constraint set before feeding the resulting optimization problem to an LP solver. Subsequently, graph based algorithms for solving the k-phase clock scheduling problem were presented in [SBS92]; graph-based algorithms for clock schedule optimization for two-phase circuitry have also been presented in [ILP92]. For a fixed clock period, the timing constraints can be represented as a set of difference constraints of the type $x_i - x_j \leq c$, where the x_i's are the variables in the problem and c is a constant. This set of constraints can be represented by a constraint graph, and the longest path in the constraint graph yields a solution to the set of difference constraints, if one exists. If the clock period is infeasible, no such solution exists, and this is determined by a procedure such as the Bellman-Ford algorithm. Therefore, a binary search over the clock period is used to find the minimum value of the clock period at which the constraint graph has a solution; this procedure works since the search space is proven to be convex in P^3. The complexity of this algorithm is approximately $O((|\Psi|k^2\log_2|\Psi|)$, where $|\Psi|$ is the cardinality of the set, Ψ, of latches in the circuit, and k is the number of phases of the clock [SBS92].

A vital contribution of [Szy92], subsequently used in many other papers such as [SR94, SD96, MS98b], is the development of the predecessor heuristic to update the value of the search boundary on the detection of a positive cycle in the constraint graph; this is described in more detail in Section 3.4.5.

2.7 TIMING ANALYSIS OF DOMINO LOGIC

Domino logic is a popular choice of design style for implementing high speed logic designs since it has the advantage of small area, fast operation and low power. In this section, we will discuss techniques used for the timing analysis of domino logic.

The basics of domino gate operation may be explained with the help of a representative domino gate configuration is shown in Figure 2.11. When the clock input is low, the gate precharges, charging the dynamic node d to logic 1. In the next half-cycle of the clock when it goes high, the domino gate evaluates, i.e., the dynamic node either discharges or retains the precharged state, depending on the values of the input signals. The two-step mode of operation with a precharge and an evaluate phase causes the timing relationships in domino logic to be more complex than those for static logic.

The clock input to the domino gate is shown in Figure 2.11. The precharge phase begins at $T_{c,f}$ and continues until $T_{c,r}$, and the evaluate phase begins at that time and ends at time $T_{c,f} + P$ where P is the period of the clock signal

feeding the domino gate. The reference time $t = 0$ is set with respect to the clock signal at the primary inputs. If more than one clock signal is used, any one of them may be used as a reference, and the transition times of the other clocks may be expressed according to the reference.

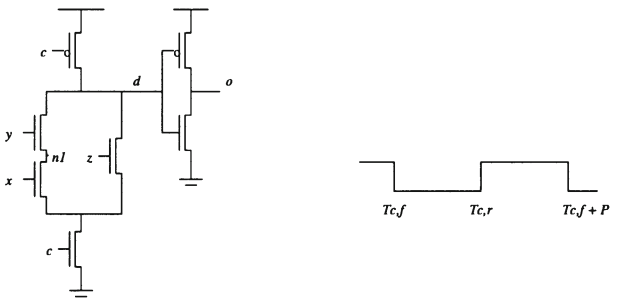


Figure 2.11. A typical domino circuit.

2.7.1 *Timing Constraints*

The node timing constraints for domino logic may be expressed in terms of the signal arrival times and the clock arrival time. Note that in case of multiple clocks for the domino logic, the clock signal c should be set to be the clock signal that feeds the gate that is currently under consideration. The set of constraints were first presented in [VCL+96] and subsequently modified in [VMS96, ZS98]; the following presentation is the most complete of all and is taken from [ZS98].
(i) The first set of constraints addresses the arrival time of a falling transition at any data input of a domino gate. Any such falling event should meet the setup-time requirement to the rising edge of the evaluate clock to ensure that the dynamic node is not inadvertently discharged by a late arriving signal. If $T_f(in)$ refers to the falling event time of the input node, then it is required that

$$T_f(in) \leq T_{c,r} - T_{setup} \tag{2.23}$$

where the setup time T_{setup} is a constant that acts as a safety margin.
(ii) The second set of constraints is related to the arrival time of a rising transition at the output of the domino gate, and is presented here in terms of the arrival time of rising transitions at each of the input nodes. The constraint may be stated as follows: the rising event of the output node of the domino gate must be completed before the falling edge of evaluate clock. If $T_r(out)$ refers to the rising event time at the output node, then the circuit operates correctly only if

$$T_r(out) \leq T_{c,f} + P \tag{2.24}$$

In other words, before the beginning of the precharge for next cycle, the correct evaluation result must have traveled to the output node.

For example, in Figure 2.11, the rising event of the output node of domino gate with output o must satisfy relation (2.24). Therefore,

$$T_r(o) = \max((T_r(x) + D_f(x,d), T_r(y) + D_f(y,d), T_r(z) + D_f(z,d), T_{c,r} + D_f(c,d))) + D_r(d,o) \quad (2.25)$$

where

$T_r(x)$, $T_r(y)$, $T_r(z)$ are the rising event times at inputs x, y and z, respectively
$D_f(i,d)$ represents the delay of a falling transition at the dynamic node d due to a rising transition at input $i \in \{x,y,z\}$
$D_r(d,o)$ represents the rise delay of the inverter feeding the gate output node o
$D_f(c,d)$ is the delay from the clock node c to the dynamic node d

Therefore, for $i \in \{x,y,z\}$,

$$D_f(i,d) + D_r(d,o) - P \leq T_{c,f} - T_r(i) \quad (2.26)$$

$$D_f(i,d) + D_r(d,o) - P \leq T_{c,f} - T_{c,r} \quad (2.27)$$

The relation (2.26) corresponds to the requirement that the rising edge of each input should appear in time for the falling edge of the evaluate clock so as to allow sufficient time for the output to be discharged.

The above formulation considers the delay from each node to the output node separately since these would typically be different. To illustrate this, it can be seen in Figure 2.11 that if the latest arriving input is y, then the fall transition does not need to discharge node n_1. On the other hand, if the latest arriving input is x, then node n_1 may have to be discharged, in addition to the output node o.

The relation (2.27) ensures that the pulse width of the evaluate clock is sufficient for pulling down the output node when the last transistor to switch is the lowermost one, connected to the clock node.

(iii) The third set of constraints addresses the timing requirements on rise transitions at the dynamic node d. The rising event d of the domino gate must be completed before the rising edge of the evaluation clock, i.e.,

$$T_r(d) \leq T_{c,r} \quad (2.28)$$

If the rise time of the dynamic node through the p-transistor fed by the clock is denoted by $D_r(c,d)$, then the rising event time can be expressed as:

$$T_r(d) = T_{c,f} + D_r(c,d) \quad (2.29)$$

This leads to the constraint given by

$$D_r(c,d) \leq T_{c,r} - T_{c,f} \quad (2.30)$$

This implies that the pulse width of precharge must be capable of pulling up the output node.

Note that unlike (ii) above, the delay to only node d is considered here, and not to the output node o. Note that the constraint to node o is very loose and is satisfied except in the most pathological cases. There are two possibilities:

- if the evaluate phase causes the dynamic node to go low, then the completion of the fall transition at o during precharge is irrelevant.
- if the evaluate phase causes the dynamic node to remain high, then it will remain so until the end of the precharge phase in the next clock cycle, and the node o correspondingly has a long time (equal to $P + T_{c,r}$) to discharge, and therefore, the constraint is very loose.

In most cases, the overriding timing constraint will be due to some other constraint, such as that corresponding to a gate at which the signal o is being utilized.

In addition to the constraints in (i) through (iii) above, there is another pair of timing constraints for domino logic.

(iv) The first of these states that in the previous clock cycle, a falling input data line may not go down to logic 0 but must be held at the logic 1 level until the output transition in that cycle has been completed. In other words, it is required that

$$T_f(in) \geq T_{c,r} - P + \max_{\forall in \in I_{domino}} (D_f(in, d) + D_r(d, o)) \tag{2.31}$$

(v) For any rising input data line, the hold constraint states that a transition should occur only after $T_{c,f}$, i.e.,

$$T_r(in) \geq T_{c,f} \tag{2.32}$$

These constraints are similar to the short path constraint in static timing analysis.

The presentations in [VCL+96, VMS96] differ from the above descripton in that they break up constraint (ii) into two separate constraints, and do not describe constraint (iv).

2.7.2 *Timing Analysis*

The timing analysis procedure described here is based on the CPM procedure. In mixed static-domino circuits, the static elements can easily be represented by nodes in the CPM graph. Dynamic nodes can also be represented as nodes in the CPM graph by representing output rise and fall times by expressions similar to equation (2.25), and then using the relaxed version of the constraint, replacing each "max" by an inequality. This may be intuitively seen by observing that the relations in Section 2.7.1 can be written in terms of difference constraints [CLR90].

The determination of rising and falling event arrival times at the output node of a domino gate is similar to the CPM computation for static gates. A major difference is that the rising event at the dynamic node is related only

to the falling edge of the domino clock and is independent of the other input nodes. This effect is captured by setting the value of D_r from each input node of the domino gate to the output node as $-\infty$. The domino clock input node is treated in the same way as any primary input node, and the rising or the falling edge of the clock provide the corresponding event times for the clock node. In case the clock input to the domino gate is the output of another gate, its event time is given by the maximum of all gate input event arrival times, plus the transition delay of the gate.

2.8 CONCLUDING REMARKS

In this chapter, we have covered the basics of timing analysis for combinational circuits, edge-triggered sequential circuits, and level-clocked circuits, and for domino circuits. We have also made small forays into the domain of timing optimization by discussing methods for minimum padding to resolve short path constraints, and for clock schedule optimization. This presentation forms the basis for our subsequent detailed treatment of two timing optimization methods, clock skew optimization and retiming, in the remainder of the book.

Notes

1. This is in contrast with an RC mesh, which is like an RC tree, except that resistive loops are permitted.

2. The vintage of the article is apparent from the fact that it illustrated the use of PERT on an example of a ferrite core system design.

3. This basic procedure is widely used in many other algorithms for clock skew optimization and retiming, presented in Chapters 3 and 4, respectively, and the reader is invited to read those sections for a fuller understanding.

3 CLOCK SKEW OPTIMIZATION

3.1 THE NOTION OF DELIBERATE CLOCK SKEW

Conventional synchronous circuit design is predicated on the assumption that each clock signal of the same phase arrives at each memory element at exactly the same time. In a sequential VLSI circuit, due to differences in interconnect delays on the clock distribution network, this simultaneity is difficult to achieve and clock signals do not arrive at all of the registers at the same time. This is referred to as a skew in the clock. In a single-phase edge-triggered circuit, in the case where there is no clock skew, the designer must ensure that for correct operation, each input-output path of a combinational subcircuit has a delay that is less than the clock period. In the presence of skew, however, the relation grows more complex and the task of designing the combinational subcircuits becomes more involved.

The conventional approach designs the clock distribution network so as to ensure zero clock skew. An alternative approach views clock skews as a manageable resource rather than a liability. It manipulates clock skews to advantage by intentionally introducing skews to improve the performance of the circuit. To illustrate how this may be done, consider the edge-triggered circuit shown in Figure 3.1, and assume each of the inverters to have a unit delay. This circuit cannot be properly clocked to function at a period of 2 time units, because as shown in the figure, the required arrival time of the signal at register F1 is 2 units, while the data arrives after 3 time units. It is readily verifiable that

N. Maheshwari et al., *Timing Analysis and Optimization of Sequential Circuits*
© Kluwer Academic Publishers 1999

the fastest allowable clock for this circuit has a period of 3 units. However, if a skew of +1 unit is applied to the clock line to register F1, the circuit can operate under a clock period of 2 units. This is possible because as shown in Figure 3.2, the application of a skew of +1 unit delays the clock arrival at register F1 by one unit, thus changing the required data arrival time to the new arrival time of the first clock tick, which is 3 units (i.e., the period of 2 unit delayed by +1 unit). Under these circumstances, the actual data arrival time of 3 units does not cause a timing violation, and the circuit is correctly clocked. A formal method for determining the minimum clock period and the optimal skews was first presented in the work by Fishburn [Fis90], where the clock skew optimization problem was formulated as a linear program that was solved to find the optimal clock period.

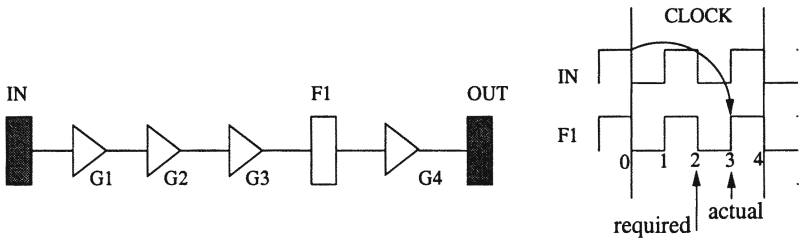


Figure 3.1. An Example Circuit.

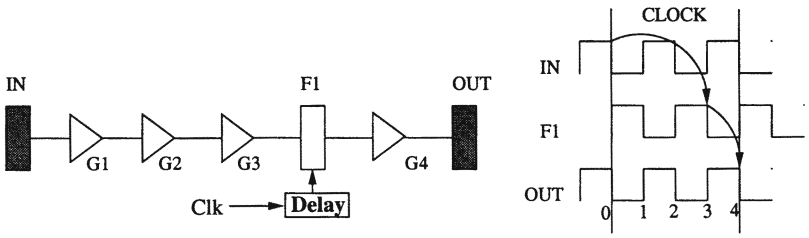


Figure 3.2. The advantage of nonzero clock skew.

The organization of material in this chapter is as follows. The method of clock skew optimization began as a controversial idea when it was first proposed, but is beginning to be accepted much more widely now; we first address this issue. Next, we briefly describe techniques that may be used to build zero-skew and fixed-skew clock trees. This is followed by clock skew optimization algorithms for period optimization, keeping the structure of the remainder of the circuit unchanged. Finally, the use of clock skew optimization in conjunction with transistor sizing, where the parameters of the circuit are changed at the same time as the optimal skews, and its use for ground bounce reduction, are presented.

3.2 IS CLOCK SKEW OPTIMIZATION SAFE?

A common misconception about changing clock skews has been that it is believed to be an "unsafe" optimization, in that a small change in the gate/interconnect delays may cause a circuit with precariously small tolerances to malfunction. In fact, this is not so; one can build in safety margins, as shown in Section 3.4 that ensure that skewing errors do not disrupt circuit functionality. These safety margins ensure that the circuit will operate in the presence of unintentional process-dependent skew variations. Introducing deliberate delays within the clocking network has been a tactic that has long been used by designers [Wag88]. From the design automation point of view, zero-skew clock tree construction techniques may be adapted to build fixed-skew clock networks. A proof of the practicality of this concept was demonstrated in a pipelined data buffer chip using the concept of skewed clocks, which was designed and fabricated in [HW96].

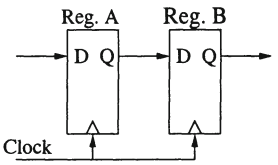


Figure 3.3. The perils of zero skew.

It is, in fact, a misconception to believe that zero skew is entirely safe. To see this, consider a shift register consisting of register A whose output is connected to register B with *no* combinational logic between the two [SSF95, SSF98], as shown in Figure 3.3. Even for a circuit designed for zero skew, a small unintentional positive skew at register B will cause double-clocking, i.e., a short path constraint violation. Such problems may be avoided by the use of the safety margins mentioned in the previous paragraph and the introduction of deliberate nonzero skew: a small amount of deliberate positive skew at A provides an effective safety margin against short path violations.

3.3 CLOCK TREE CONSTRUCTION

3.3.1 Early Attempts

The clock distribution network is required to propagate the clock signal to every register in a synchronous circuit. Consequently, it must be designed carefully to ensure the optimal use of resources and to achieve performance requirements on the clock signal at the sink nodes. These performance requirements include requirements on the signal arrival time and on the sharpness of the clock edge. The skew constraint on clock networks is a hard constraint, and to ensure the tractability of the problem, many clock networks are constructed as trees. In this section, we will present the essentials of clock tree construction methods targeted at achieving zero skew and fixed skew. Although they will not be discussed here, it is worth pointing out that meshes have been proposed for

clock distribution, for example, in [XK95, DCJ96]. This section is only intended to be an abbreviated reference on clock tree construction; for details, the reader is referred to [Fri95, KR95, CHKM96].

Early methods for clock tree design attempted to achieve zero-skew by balancing the wire length to each sink. H-trees [FK82], illustrated in Figure 3.4 and so named because of their structure, were recognized for years as a technique to help reduce the skew in synchronous systems. The H-tree ensured, by construction, that all sinks of the clock tree are equidistant from the clock source. For regular structures such as systolic arrays, the H-tree works well to reduce skew, but in the general case, asymmetric distributions of clock pins are common and the H-tree may not be effective for clock routing. To address this issue, the method of recursive geometric matching [KCR91] was proposed, and is an extension of the H-tree idea to structures with a smaller degree of regularity.

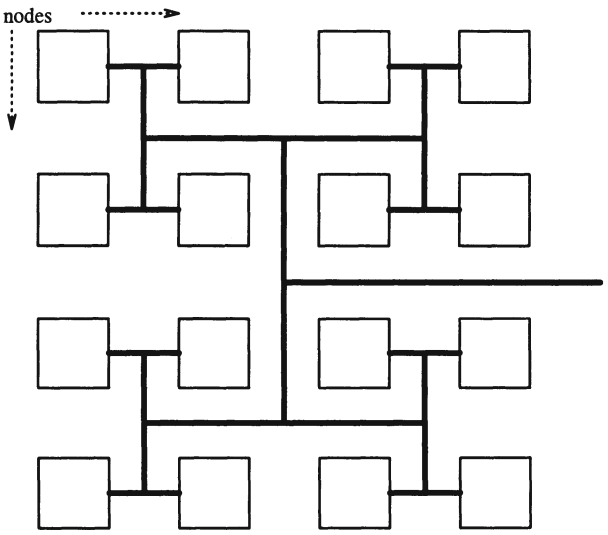


Figure 3.4. H-tree layout for clocking.

In contrast with the bottom-up approach used in both of these methods, the method of means and medians [JSK90] works in a top-down manner, recursively bipartitioning the routing area into subregions. The centers of these regions are connected appropriately, with the aim of generating at a balanced tree in which the distance from the clock source to the sinks is minimized.

3.3.2 Exact Zero-skew Clock Routing

Each of the methods described so far simplifies the problem of equalizing the clock delays into the geometric problem of equalizing the path lengths on a Manhatan grid. This assumption is valid for a perfectly symmetric clock network in which the load capacitances at all the sinks is precisely the same; however, this

is an invalid assumption for real circuits. The work in [Tsa91, Tsa93] recognized that due to uneven loading and buffering effects, path-length equalization methods do not achieve the effect of balancing clock delays. An approach that incorporates this effect and uses the Elmore delay model to automatically build zero-skew clock trees was suggested. We will now describe this method in detail, and explain how it may be extended to build clock trees with specified, and possibly nonzero, skews at the sink nodes.

The Elmore delay model [RPH83] is used to calculate the signal propagation delay from a clock source to each clock sink. A modified hierarchical method is proposed for computing these delays in a bottom-up fashion. This algorithm takes advantage of the structure of the Elmore delay calculations to construct a recursive bottom-up procedure for interconnecting two zero-skew subtrees to form a new tree with zero skew.

We now describe one recursive step of the bottom-up procedure. In each such step, two subtrees are combined into one subtree; by construction, it is ensured that each such subtree has zero skew from its current root to all of its sinks. In other words, the signal delay from the root to the leaf nodes of the subtree are forced to be equal. In the first step, each sink node constitutes a subtree, and the skew within each such subtree is trivially seen to be equal to zero. In subsequent steps, partially constructed zero-skew subtrees are combined.

In one step of the recursion, two zero-skew subtrees are connected with a wire and a new root is defined for this combined structure, ensuring zero skew from the root to all sinks in the merged tree. This is illustrated in Figure 3.5, and this new root is referred to as the *tapping point.* If we denote the total wire length of this connecting wire segment as l, and the wire length from the tapping point to the root of subtree1 be $x \times l, 0 \leq x \leq 1$, then the wire length from the tapping point to the root of subtree2 is $(1-x) \times l$.

A closed-form expression for the value of x that yields zero skew will now be determined. Consider each subtree in Figure 3.5. Since subtree1 [subtree2] has zero skew, the delay from its root to each leaf node is equal, say t_1 [t_2]. Let the corresponding total capacitances in the subtrees be C_1 [C_2], respectively. The wires of length $x \times l$ and $(1-x) \times l$ are modeled using the RC π model shown in the figure.

The delay from the tapping point to each leaf node in subtree1 is equated to the delay to each node in subtree2 using the Elmore delay formula [SK93] to provide the following equation

$$r_1(\frac{c_1}{2} + C_1) + t_1 = r_2(\frac{c_2}{2} + C_2) + t_2 \tag{3.1}$$

Substituting $r_1 = \alpha x l$, $r_2 = \alpha(1-x)l$, $c_1 = \beta x l$, and $c_2 = \beta(1-x)l$, where α and β are, respectively, the per unit resistance and capacitance of the interconnecting wire, the tapping point is found to correspond to

$$x = \frac{(t_2 - t_1) + \alpha l \left(C_2 + \frac{\beta l}{2}\right)}{\alpha l(\beta l + C_1 + C_2)} \tag{3.2}$$

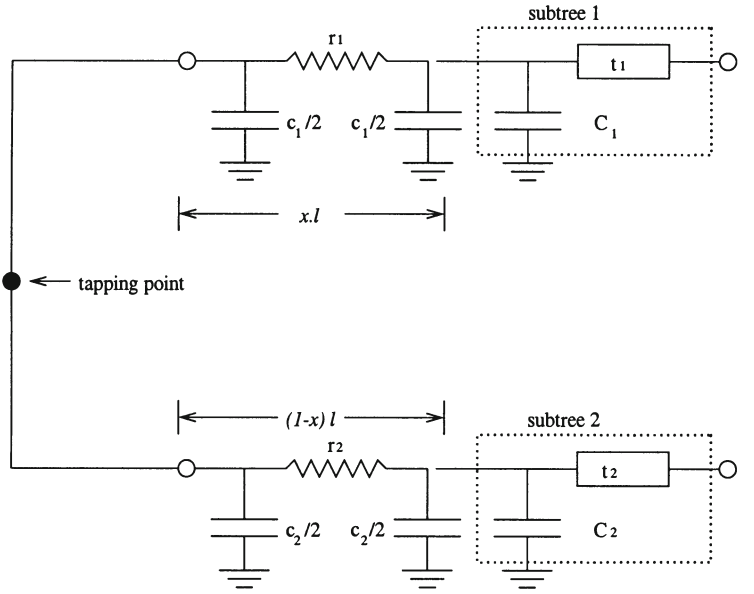


Figure 3.5. Zero-skew merge of two subtrees [Tsa93] ©1993 IEEE.

If the solution of equation (3.2) results in a value x that satisfies $0 \leq x \leq 1$, then the tapping point lies along the segment interconnecting the two subtrees and is legal. In case x lies outside this range, it implies that the loads on the two subtrees are too disparate. If so, the tapping point is set to lie at the root of the subtree with the larger loading, and the wire connected to the other segment is elongated in order to ensure that its additional delay balances the delay of the first subtree. This procedure of elongating the wire is referred to as *snaking*. If $x < 0$, then the tapping point is kept at $x = 0$ and the connecting wire is elongated by an additional length l'. The value of l' is found by matching the delays in each subtree, and is given by

$$l' = \frac{\sqrt{(\alpha C_1)^2 + 2\alpha\beta(t_2 - t_1)} - \alpha C_1}{\alpha\beta} \tag{3.3}$$

The formula for l' for the $x > 1$ scenario can be derived in a similar manner.

Thus, in this approach the delay time to each node is derived from its immediate predecessor, the branch resistance, and the subtree capacitance. Buffering effects are accounted for in this procedure by storing C_1 and C_2, as the total left and right subtree capacitances, respectively, up to their closest downstream buffer locations; buffers are added to ensure that the clock edge remains sharp.

3.3.3 Further Refinements

The procedure described by the algorithm described above does not attempt to reduce the length of the zero-skew clock tree. The deferred-merge embedding

(DME) algorithm [Eda91, BK92, CHH92] substantially reduces the tree cost while satisfying the skew specifications. In each step of the zero-skew merging procedure described earlier, the two subtrees are connected by a wire of length l. In a Manhattan geometry, there are several routes between two points that have the exact same length. Therefore, the precise route taken by the wire connecting the two zero-skew subtrees is important in that it affects the optimality (but not the correctness under Elmore delays) of future merge operations, where the optimality is defined in terms of minimizing the wire length of the clock tree.

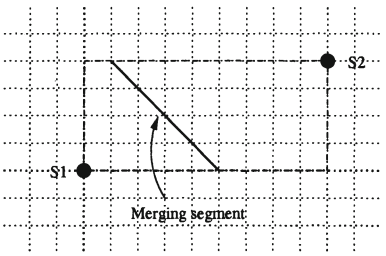


Figure 3.6. A merging segment between two points, with $x = \frac{5}{13}$.

The DME procedure works by deferring the precise route, or embedding, corresponding to each node for as long as possible, and operates in two phases. In the first bottom-up phase, a tree of "merging segments" is constructed, where each merging segment is the locus of possible placements of a specific tapping point in the clock tree, under a Manhattan geometry. An example of a merging segment for two points, S1 and S2, is shown in Figure 3.6, with the value of x in equation (3.2) set to $\frac{5}{13}$. It should be pointed out that the locus of points that form a merging segment for a pair of straight lines segments is another straight line segment; this implies that as the method goes up the recursive merging process, the locus of merging points is always given by a straight line. In the second top-down phase, the exact location of these internal nodes are resolved with the purpose of minimizing the total wire length of the clock tree.

Apart from reducing the delay by minimizing the wire lengths and inserting buffers, one may also use wire width optimization for sharpening the clock edge. Techniques for this purpose have been presented in, for example [Eda93, PMP93, PMOP93]. The problem of optimally inserting buffers in clock trees has been addressed in [CS93b, VMS95, TS97].

The zero-skew merge procedure is extended to RLC clocking networks in [LS96, LS98], using higher-order moment matching methods that are a generalization of the Elmore delay modeling approach. The procedure ensures adequate signal damping and satisfies constraints on the clock signal slew rate.

3.3.4 Fixed Skew Clock Network Construction

The procedures described above can be extended [Tsa93] to build a clock tree that is required to deliver the clock with a predetermined skew at each

sink. This problem is termed the fixed-skew clock network construction, or the prescribed-skew routing problem. As described earlier in this chapter, the major utility of nonzero skews is to enable time-borrowing between combinational circuit segments that are connected by a flip-flop.

The technique of [Tsa91, Tsa93] (and its extensions) can be adapted to solve this problem by adding a fictitious delay element on each clock sink. For a prescribed skew of $+D_i$ at the ith element, the algorithm begins with each sink node corresponding to a (trivial) tree with a delay of $-D_i$, instead of zero as before. Note that according to the notation, a positive skew $+s$ implies that the clock signal is delayed by time s.

Therefore, in principle, in terms of the design automation effort, it is no harder to design fixed-skew clock trees than it is to build zero-skew trees. Several other papers on fixed-skew clock network construction have been published, including [NF96, NF97, HLFC97]. The work in [NF96, NF97] develops a technique that first determines the optimal skew schedule for a circuit, and then constructs a fixed skew network by using the delays of wires and by inserting buffers for additional delays, whle minimizing sensitivity to process and environmental delay variations. The approach in [HLFC97] is distinguished by the fact that it performs a detailed circuit-level design and proves the viability of the process through a fabricated chip.

3.4 CLOCK SKEW OPTIMIZATION

It has been demonstrated at the beginning of this chapter that the use of intentional nonzero clock skews can speed up the clock period for a sequential circuit since it facilitates cycle-borrowing. The work of Fishburn [Fis90] formalizes an approach to find the optimal skews at each memory element in an edge-triggered circuit, where the optimality is defined in terms of minimizing the clock period. The problem is posed as a linear program, and this formulation is described in this section.

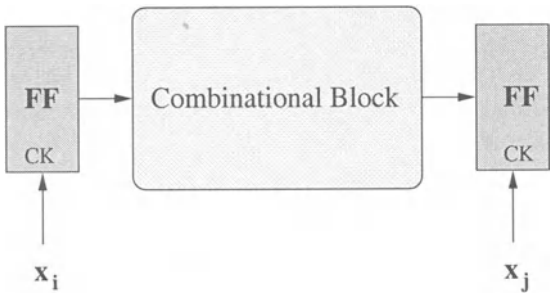


Figure 3.7. A combinational block in a sequential circuit.

3.4.1 *Timing Constraints*

Consider a combinational block in a sequential circuit, as shown in Figure 3.7. Let FF_i and FF_j be a pair of flip-flops at the input and output, respectively, of the combinational block, with skews of x_i and x_j, respectively.

Denote the delay of the combinational block between them as $d(i,j)$, with $\underline{d}(i,j)$ being the minimum delay and $\overline{d}(i,j)$ being the maximum delay). If T_{hold} is the flip-flop hold time, T_{setup} the flip-flop setup time, and P the clock period, then in the presence of skews the double-clocking and zero-clocking constraints may be written as described below, in a manner similar to the zero-skew case discussed in Section 2.3.

Double-clocking/Short path constraint To avoid double-clocking, the data from the next clock cycle should arrive at FF_j no earlier than a time T_{hold} after the current clock. The data leaves FF_i at time x_i and, therefore, the earliest time it can reach FF_j is $x_i + \underline{d}(i,j)$. Thus, the double-clocking constraint illustrated in Figure 3.8 is

$$x_i + \underline{d}(i,j) \geq x_j + T_{hold} \tag{3.4}$$

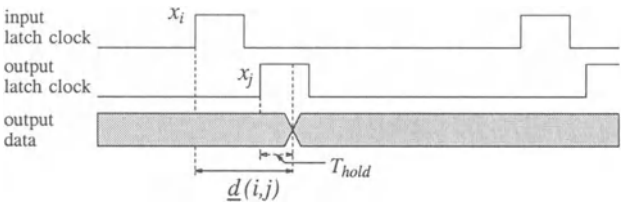


Figure 3.8. Double-clocking.

Zero-Clocking/Long path constraint To avoid zero-clocking, the data from the current clock cycle should arrive at FF_j no later than a time T_{setup} before the next clock. Since the data leaves FF_i at time x_i, the latest time by which it will reach FF_j is $x_i + \overline{d}(i,j)$. The zero-clocking constraint, shown in Figure 3.9, may be expressed as

$$x_i + \overline{d}(i,j) \leq x_j + P - T_{setup} \tag{3.5}$$

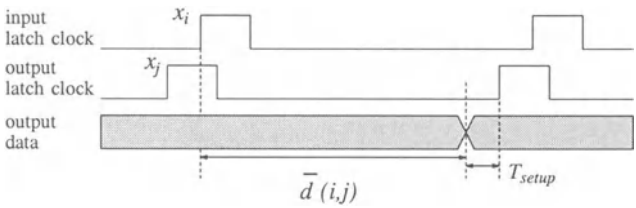


Figure 3.9. Zero-clocking.

3.4.2 Clock Period Minimization

The problem of minimizing the clock period P by controlling the clock skew at each flip-flop, subject to correct timing, can now be formulated as the following linear program

$$\begin{array}{lll} \text{minimize} & P & \\ \text{such that} & x_i + \underline{d}(i,j) & \geq x_j + T_{hold} \\ & x_i + \overline{d}(i,j) & \leq x_j + P - T_{setup} \end{array} \tag{3.6}$$

The linear program assumes that the delays of the combinational segments are constant. These delays can be calculated efficiently by using the method of Section 2.2.2.

The above formulation may be altered to model uncertainties in the clock skews. If it can be guaranteed that in the manufactured circuit, the skew at flip-flop k will be within the range $[x_k - \delta/2, x_k + \delta/2]$, where x_k is the designed value of the skew, then the difference between any skews, $(x_i - x_j)$ in the manufactured circuit, must be within δ of the designed value of $(x_i - x_j)$. A linear program that is guaranteed to produce a functional manufactured circuit is formulated below:

$$\begin{array}{lll} \text{minimize} & P & \\ \text{such that} & x_i + \underline{d}(i,j) & \geq x_j + T_{hold} + \delta \\ & x_i + \overline{d}(i,j) & \leq x_j + P - T_{setup} - \delta \end{array} \tag{3.7}$$

An alternative formulation to maximize the tolerance of the solution to unpredictable changes, for a given clock period P, is as follows. This may be achieved by maximizing the minimum slack over all the constraints, converting the problem into a minmax problem. A new variable M is introduced, and is added to each of the main constraint inequalities, so that maximizing M is tantamount to finding the skew values that maxime the minimum slack over all the inequalities. The precise formulation is as shown below:

$$\begin{array}{lll} \textit{maximize} & M & \\ \textit{s.t.} & x_i + \underline{d}(i,j) - M & \geq x_j + T_{hold}, \\ & x_i + \overline{d}(i,j) + T_{setup} + M & \leq x_j + P \end{array} \tag{3.8}$$

3.4.3 Efficient Solution of Skew Optimization Problem

The constraints of the linear program (3.7) are rewritten as

$$\begin{array}{lll} x_i - x_j & \geq & T_{hold} - \underline{d}(i,j) + \delta \\ x_j - x_i & \geq & T_{setup} + \overline{d}(i,j) - P + \delta, \end{array} \tag{3.9}$$

Note that the skews at the primary inputs and the primary outputs may be set to zero under the assumption that they cannot be controlled or adjusted.

For a constant value of P, the constraint matrix for this problem reduces to a system of difference constraints [CLR90]. For such a system, it is possible to construct a constraint graph. The vertices of the constraint graph correspond to the x_i variables, and each constraint $x_a - x_b \geq c$ corresponds to a directed edge from node b to node a with a weight of c. If P is achievable, then a set of skews that satisfies P can be found by solving the longest path problem on this directed graph. The clock period P is feasible, provided the corresponding constraint graph contains no positive cycles.

This observation was utilized in [DS94] for efficient solution of the skew optimization problem. The optimal clock period is obtained by performing a binary search. At each step in the binary search (for clock period P) the Bellman-Ford algorithm, to be described shortly, is applied to the corresponding constraint graph to check for positive cycles. The procedure continues until the smallest feasible clock period is found. The following skeletal pseudocode is used to perform the binary search for the optimal clock period.

```
Algorithm MINPERIOD_WITH_OPTIMAL_SKEWS
      Construct the constraint graph;
      P_min = P_low;          ...(Lemma 3.3 below)
      P_max = P_high;         ...(Lemma 3.4 below)
      while (P_max - P_min) > ε {
         P = (P_max + P_min)/2;
         if constraint graph has a positive cycle
            P_min = P;
         else
            P_max = P;
      }
```

The following results are used to show the correctness of the binary search; for proofs, the reader is referred to [DS94, SD96].

Lemma 3.1 : If $P = P_1$ does not permit a feasible solution to the linear program (3.6), then nor does $P = P_2 < P_1$.

Corollary 3.2 : If $P = P_1$ permits a feasible solution to the linear program (3.6), then so does $P = P_2 > P_1$.

Next, two lemmas that provide bounds P_{low} and P_{high} on the optimal clock period, are presented.

Lemma 3.3 : Given a pair of inequalities in relation (3.9), define

$$P_{ij} = \overline{d}(i,j) - \underline{d}(i,j) + T_{setup} + T_{hold} + 2\delta \tag{3.10}$$

A lower bound, P_{low}, on the value of the optimal clock period is given by

$$P_{low} = \max_{all(i,j)} P_{i,j} \geq 0. \tag{3.11}$$

where the max operates on all pairs (i, j) of FF's that are connected by a combinational path.

Lemma 3.4 : If the original circuit satisfies all short path constraints, then an upper bound, P_{high} on the value of the optimal clock period is given by

$$P_{high} = \max_{all(i,j)} (\bar{d}(i,j) + T_{setup} + \delta) < \infty \tag{3.12}$$

As before, the max operates on all pairs (i, j) of FF's that are connected by a combinational path.

3.4.4 Applying the Bellman-Ford Algorithm

As stated earlier, each step of the binary search applies the Bellman-Ford algorithm to solve the longest path problem on the constraint graph to determine whether P is feasible or not, and if so, to find the values of the corresponding skews. A brief description of the procedure follows.

The Bellman-Ford algorithm may be applied to solve a single-source longest path problem on a directed graph. Unlike Dijskstra-like algorithms, it can handle edge weights of either sign, but has a higher computational complexity. The pseudocode for the application of the algorithm to a graph $G = (V, E, \mathbf{w})$, where V and E are the set of vertices and edges, respectively, of the graph, and $\mathbf{w}$ is the vector of edge weights, is as follows:

```
Algorithm BELLMAN_FORD
      for each vertex v ∈ V
          x_v ← 0;
          predecessor(v) ← ∅;
      for i ← 1 to |V|-1
          if x_v < x_u + w(u,v) then
             x_v ← x_u + w(u,v);
             predecessor(v) ← u;
      for each edge (u,v) ∈ E
          if (x_v < x_u + w(u,v)) then
             /* A positive cycle exists */
             SATISFIABLE ← FALSE;
          else
             SATISFIABLE ← TRUE;
```

If the skews are initialized to 0, the Bellman-Ford solution achieves the objective of minimizing $|x_{i,max} - x_{i,min}|$ [CLR90]. On a graph with V vertices and E edges, the computational complexity of this algorithm is $O(V \cdot E)$.

In the solution found above, all skews must necessarily be positive, since the weight of each node in the Bellman-Ford algorithm was initialized to zero. Also, in general, the skew at the host node (corresponding to primary inputs and outputs) could be nonzero at the end of these iterations. Since the objective is to ensure a zero skew at the primary input and output nodes, the solution

is modified to achieve this. Note that if $[x_1, \cdots x_n]$ is a solution to a system of difference constraints in $\mathbf{x}$, then so is $\mathbf{x}' = [(x_1 + k), (x_2 + k), \cdots, (x_n + k)]$. Therefore, by selecting k to be the negative of the skew at the host node, a solution $\mathbf{x}'$ with a zero skew at the host is found.

3.4.5 *Complexity Analysis and Efficiency-enhancing Methods*

The number of iterations of the binary search, in the worst case, is $(P_{high} - P_{low})/\epsilon$. The time required to form the constraint graph may be as large as $|f| \cdot |G|$, where $|f|$ is the maximum number of inputs to any combinational stage. In practice, though, it is seen that this upper bound is seldom achieved. Therefore, the iterative procedure above, when carried to convergence, provides the solution to the linear program (3.7) with a worst-case time complexity of

$$O\left(F \cdot E \cdot log_2\left[(P_{high} - P_{low})/\epsilon\right] + |f| \cdot |G|\right) \tag{3.13}$$

where F is the number of flip-flops in the circuit, E is the number of pairs of flip-flops connected by a combinational path, P_{high} and P_{low} are as defined in Lemmas 3.3 and 3.4, and ϵ, defined in the pseudocode above, corresponds to the degree of accuracy required.

We caution the reader that the complexity shown above is not a genuine indication of the complexity for real circuits. Firstly, in practical cases, it is seen that $E = O(F)$. Secondly, if the implementation is cleverly carried out, using back-pointers during the Bellman-Ford process, to be described in the remainder of this section, and the procedure in Section 2.2.3 for $\overline{d}(i, j)$ calculations, the computational cost can be substantially reduced.

Instead of dividing the interval into half during each step of the binary search, it is possible to use circuit information to reduce the computation and update the search space more efficiently [Szy92]. Suppose that for some value of P, the constraint graph was found to be infeasible. This implies that a positive cycle exists, and each edge on that positive cycle has either a constant weight, if it corresponds to a short-path constraint, or a weight of the form (constant - P), if it is a long-path constraint edge. Given any positive cycle, it is possible to sum up these weights on the edges and update the bound on P so that any value of P that is less than this bound would force this cycle to be infeasible. This reduces the binary search space.

The detection of a positive cycle proceeds as follows. During the Bellman-Ford procedure, a back-pointer `predecessor(`v`)` is maintained from each vertex v to the predecessor vertex that updated its value; in other words, the edge between these vertices corresponds to an active constraint. After the Bellman-Ford procedure is completed, if the constraint graph is infeasible, a reverse trace is carried out from the vertex v for which it was found that $x_v > x_u + w(u, v)$. This node v must necessarily lie on a positive cycle. The procedure steps back to the predecessor of v, and further back, until v is reached again; at this time, a positive cycle in the graph has been detected. The sum of the weights of all edges on this cycle is calculated as a function of P. As stated in the previous paragraph, this sum is a linear function of P of the form $k_1 - k_2 P$; the lower

bound on the clock period is updated to k_1/k_2 to ensure that this cycle will not have a positive weight. This procedure can be repeated on every cycle in the graph of predecessor edges; note that the number of edges in such a graph is $O(|V|)$, where $|V|$ is the number of vertices, which means that this can be performed inexpensively. Similarly, if the clock period is feasible, the graph of predecessor edges may be used to update the upper bound on the clock period by exploring the amount by which the clock period could be lowered before the predecessor graph would be altered.

Note that the solution to the problem of clock period minimization corresponds to the maximum P over all positive cycles; it is, in general, not possible to find this maximum in polynomial time since the number of cycles could be exponential in the number of vertices. However, it has been seen that from a practical standpoint, the back-pointer procedure works well in reducing the amount of computation.

3.5 CLOCK SKEW OPTIMIZATION WITH TRANSISTOR SIZING

In the discussion so far, it has been assumed that the delay of each combinational path is fixed. However, the drive strength of gates can be altered by transistor sizing [SK93], thereby changing the combinational path delays. Treating clock skew optimization and transistor sizing as separate optimizations is liable to lead to a suboptimal solution, and a synergistic blend of the two procedures can greatly improve the cost-performance tradeoff for the circuit.

To illustrate this, consider a pair of combinational stages separated by latches, as shown in Figure 3.10(a). We will consider area as the cost function here, but the same idea is valid for other correlated cost functions such as power. If only the sizing transformation were to be applied, the nature of the cost-delay tradeoff curve for each stage would be as shown in Figure 3.10(b). If the timing specification on Stg_1 were stringent, then the optimal sizing solution would lie beyond the knee of the curve, at a point such as point A. Similarly, if the specifications on Stg_2 were loose, then the optimal solution might lie at a point such as C. The cost of the sizing solution would then be [Area(A) + Area(C)][1].

Now consider a situation where a small skew, S, is applied to the registers between the two stages, allowing Stg_1 to borrow time from Stg_2. This would lead to a looser specification on Stg_1, bringing the solution point from A to B, and a tighter specification for Stg_2, moving its solution from C to D. The cost of the overall sizing solution, [Area(B) + Area(D)], can be seen to be much smaller than the cost with zero skew, even for a small amount of cycle borrowing, since the cost at the solution point A was well above the knee of the curve[2]. Therefore, the judicious use of sizing in conjunction with deliberate skew can deliver solutions of significantly better quality than sizing alone. It is important to perform the two optimizations concurrently, rather than sequentially, to fully incorporate the mutual interactions between skew and sizing. This section presents three techniques for simultaneous optimization of skews and transistor sizes.

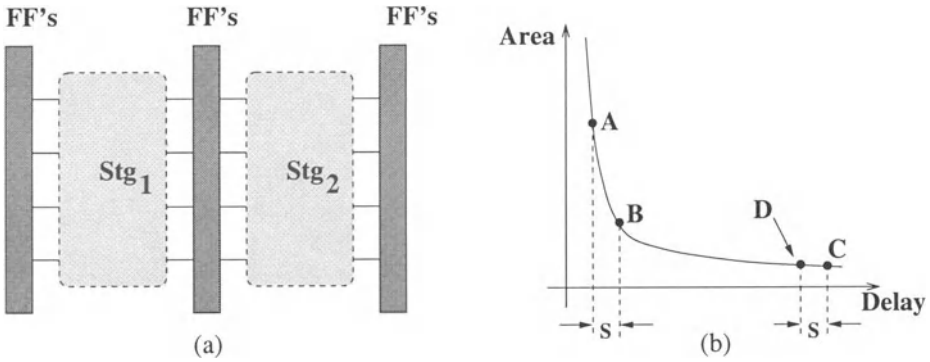


Figure 3.10. Example illustrating the importance of combining sizing and skew optimization.

3.5.1 A Linear Programming Formulation

Problem Formulation. The set of long path constraints in the presence of clock skews can be seen from inequation (3.5) to be linear in the skew variables and the gate delays. If the gate delays could also be represented in terms of linear functions of the transistor sizes, then the set of timing constraints on a circuit would be linear. This idea forms the basis of the linear programming formulation [CSH93, CSH95] discussed in this section.

The delay of a gate is represented as a convex piecewise linear function of its own size and that of its fanout gates. The delay $D(w_k, w_{k,1}, \cdots, w_{k,fo(k)})$ of a gate with size w_k, and with fanout gates of sizes $w_{k,1} \cdots w_{k,fo(k)}$, can be represented using a convex piecewise linear function with q regions as:

$$D(w, w_{k,1}, \cdots, w_{k,fo(k)}) = \begin{cases} a_1 \cdot w + b_{1,1} \cdot w_{k,1} + \cdots + b_{1,f} w_{k,fo(k)} + c_1 \\ \qquad (w, w_{k,1} \cdots w_{k,fo(k)}) \in \text{Region } \mathbf{R}_1 \\ a_2 \cdot w + b_{2,1} \cdot w_{k,1} + \cdots + b_{2,f} w_{k,fo(k)} + c_2 \\ \qquad (w, w_{k,1} \cdots w_{k,fo(k)}) \in \text{Region } \mathbf{R}_2 \\ \vdots \\ a_q \cdot w + b_{q,1} \cdot w_{k,1} + \cdots + b_{q,f} w_{k,fo(k)} + c_q \\ \qquad (w, w_{k,1} \cdots w_{k,fo(k)}) \in \text{Region } \mathbf{R}_q \end{cases} \tag{3.14}$$

$$= \max_{1 \le j \le q} (a_j \cdot w + b_{j,1} \cdot w_{k,1} + \cdots b_{j,f} w_{k,fo(k)} + c_j)$$
$$\forall (w_k, w_{k,1} \cdots w_{k,fo(k)}) \in \bigcup_i \mathbf{R}_i. \tag{3.15}$$

The second equality follows from the first since $D(w_k, w_{k,1}, \cdots, w_{k,fo(k)})$ is convex.

The variables m_k^i (p_k^i) are introduced to represent the longest (shortest) delay from each flip-flop i to gate k. The introduction of the m and p intermediate variables ensure that the number of constraints depends on the number of gates rather than the number of combinational paths; the number of constraints in the latter case would have been prohibitively large. As before, the skew at flip-flop i is denoted as x_i, while w_k and d_k are the width and delay of gate k respectively. The value of the skew is set to a constant (typically, 0) for primary inputs and outputs. The skew optimization and sizing problem for a general synchronous sequential circuit can now be formulated as follows.

$$
\begin{array}{lll}
\text{minimize} & \displaystyle\sum_{k=1}^{\mathcal{N}} \gamma_k \cdot w_k & \\
\text{subject to} & d_k \geq D(w_k, w_{k,1}, \ldots w_{k,fo(k)}), & 1 \leq k \leq \mathcal{N} \\
& w_k \geq \text{Minsize}(k), & 1 \leq k \leq \mathcal{N} \\
& w_k \leq \text{Maxsize}(k), & 1 \leq k \leq \mathcal{N} \\
& \text{For all flip-flops } i, & 1 \leq i \leq \mathcal{L} \\
& \quad x_i + p_k^i \geq x_j + T_{hold} & 1 \leq j \leq \mathcal{L}, \;\; k = Fanin(FF_j) \\
& \quad x_i + T_{setup} + m_k^i \leq x_j + P_{spec} & 1 \leq j \leq \mathcal{L}, \;\; k = Fanin(FF_j) \\
& \quad \text{For all gates } \; k = 1, \cdots, \mathcal{N} & \\
& \quad\quad m_l^i + d_k \leq m_k^i, & \forall\, l \in Fanin(k) \\
& \quad\quad p_l^i + d_k \geq p_k^i, & \forall\, l \in Fanin(k)
\end{array}
\tag{3.16}
$$

Here, γ_k is the *area coefficient*, a constant associated with gate k, so that if gate k has size w_k, then its area is $(\gamma_k \cdot w_k)$. In the notation used above, $\mathcal{N}$ is the number of gates, $\mathcal{L}$ is the number of flip-flops in the circuit, and $Fanin(i)$ is the set of gates or flip-flops at the fanin of a gate or flip-flop i.

The problem formulation above is a linear program in the variables d_i, m_i, p_i, s_i and w_i. The entries in the constraint matrix are very sparse, which makes the problem amenable to fast solution by sparse linear programming approaches.

Reducing the Number of Constraints. For large circuits the number of constraints in the above linear program can be tremendous. A symbolic propagation method is used to prune the number of constraints by a judicious choice of the intermediate variables, m and p, without sacrificing accuracy. Although the actual reduction is dependent on the structure of the circuit, experimental results showed that this algorithm can reduce the number of constraints to less than 10% of the original number for most tested circuits in [CSH95].

We begin by counting the number of linear constraints in (3.16), ignoring the constraints on the maximum and minimum sizes of each gate since these are handled separately by the simplex method. The d_k inequalities impose q constraints for each of the gates in the circuit to the linear programming formulation, where q is given in (3.15). Let $\mathcal{F} = \sum_{i=1}^{\mathcal{N}} Fanin(i)$. Then for each flip-flop i, there are $O(\mathcal{F} + \mathcal{L})$ constraints, where $\mathcal{L}$ is the total number of flip-flops in the circuit. Therefore, the total number of constraints could be as large as $O(\mathcal{N} \cdot q + \mathcal{L} \cdot (\mathcal{F} + \mathcal{L}))$. Assume that the average number of fanins to a gate is 2.5 and $q = 5$. Then $\mathcal{F} = 2.5\mathcal{N}$, and $\mathcal{L} \cdot \mathcal{F}$ is the dominant term in the expression above. For real circuits, $\mathcal{L}$ is large, and hence, the number of constraints could be tremendous. The symbolic constraint propagation method presented here prunes the number of constraints by attempting to minimize the intermediate variables m and p, without sacrificing the exactness of the original formulation. For any primary input or flip-flop, the m and p variables are introduced only for those gates that lie in that primary inputs fanout cone. Moreover, constraints on chains of gates are collapsed wherever possible by introducing a new m or p variable only when essential. The pseudocode for the procedure is shown below:

```
Algorithm SYMBOLIC_PROPAGATION()
for i = 1 to L {
   ∀ gates j, w_j ← 0; mstring(j) ← ""; pstring(j) ← "";
   for j = 1 to max_level {
     for each gate k at level j {
       if (w_l = 0 ∀ l ∈ fanin(k)) continue; /* do nothing */
       if (exactly one w(l) = 1 for l ∈ fanin(k)) {
         mstring(k) ← mstring(l) + "d_k";
         pstring(k) ← pstring(l) + "d_k";
         w_k ← 1;
       }
       else {
         mstring(k) ← "m_k^i";
         pstring(k) ← "p_k^i";
         w_k ← 1;
         for all w_l = 1, l ∈ fanin(k) {
           write out the constraints
            mstring(l) + d_k ≤ m_k^i;
            pstring(l) + d_k ≤ p_k^i;
         }
       }
     }
   }
}
```

The sequential circuit is first levelized. For this purpose, the inputs of flip-flops are considered as pseudo-primary outputs and the outputs of flip-flops

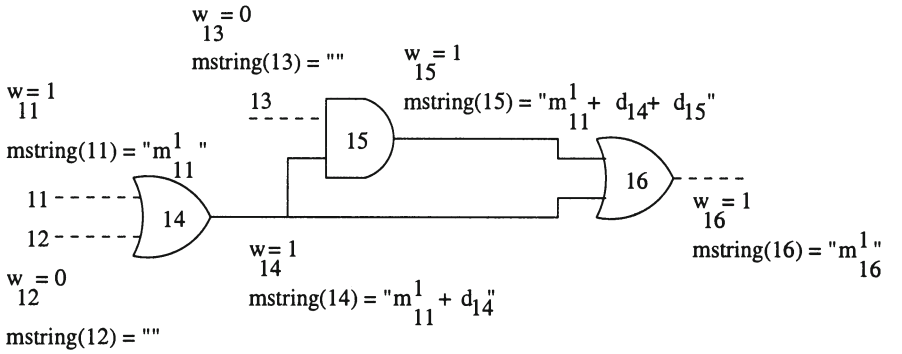


Figure 3.11. An example illustrating symbolic delay propagation algorithm.

are considered as pseudo-primary inputs. Two string variables, $mstring(i)$ and $pstring(i)$, are used to store the long-path delay and short-path delay constraints associated with gate i, respectively. For each gate and each primary input (including pseudo-primary inputs), an integer variable $w_i \in \{0, 1\}$ is introduced to indicate its status. w_i has the value 1 whenever $mstring(i)$ and $pstring(i)$ are non-empty, i.e., when the constraints stored in $mstring(i)$ and $pstring(i)$ must be propagated; otherwise, $w_i = 0$.

In the following discussion of the algorithm, we elaborate on the formation of *mstring*; the formation of *pstring* proceeds analogously. For each gate j, w_j and $mstring(j)$ are initialized by setting $w_j = 0$, and $mstring(j)$ to the null string. If $w_l = 0$ for all $l \in fanin(k)$, i.e., if all of gate k's input gates have a null *mstring*, then no constraints need to be generated or propagated. If exactly one of all of gate k's input gates, say gate l, has a non-empty *mstring*, and all others are have null *mstring*'s, then the constraint is propagated, rather than introducing a new m or p variable. This is implemented by concatenating $mstring(l)$ and "d_k", and storing the resulting string in $mstring(k)$. Also, w_k is set to 1 to indicate that further propagation is required beyond this gate. Finally, if more than one of gate k's input gates have a non-empty *mstring*, then a new intermediate variable, m_k^i is added, and the string "m_k^i" is stored at $mstring(k)$. For each input gate whose *mstring* is non-empty ($w_l = 1$), a delay constraint is needed.

Example: Figure 3.11 gives an example that illustrates the symbolic delay constraint propagation algorithm for calculating *mstring* constraints only. Assume that $mstring(11)$ = "m_{11}^1", and $mstring(12) = mstring(13)$ = "", the null string. Since gate 14 has only one input with a w value of 1, this implies that $mstring(14)$ = "$m_{11}^1 + d_{14}$" and $w_{14} = 1$. Propagating this further, it is found that similarly, $mstring(15)$ = "$m_{11}^1 + d_{14} + d_{15}$", and $w_{15} = 1$. Finally, for gate 16, since more than one input has a w value of 1, a new variable, m_{16}^1, must be introduced and w_{16} is set to 1. The following constraints may now be written and appended to the set of constraints for the linear program:

$$m_{11}^1 + d_{14} + d_{15} + d_{16} \leq m_{16}^1$$

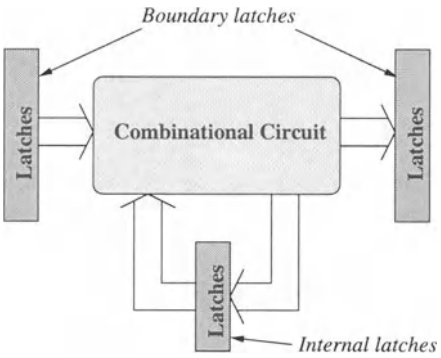


Figure 3.12. A model of a synchronous sequential circuit.

$$m_{11}^1 + d_{14} + d_{16} \leq m_{16}^1$$

Partitioning Large Sequential Circuits. The number of constraints in the linear programming formulation is proportional to the product of the number of gates and the number of flip-flops in the circuit in the worst case. Ideally for a given sequential circuit, all variables and constraints should be considered together to obtain an optimal solution. However, for large sequential circuits, the size of the linear program could become prohibitive, even after applying the pruning methods embedded in the symbolic constraint propagation algorithm. Therefore, it is desirable to partition large sequential circuits into smaller, more tractable subcircuits, so that the algorithms can be applied to each subcircuit. While this would entail some loss of optimality, an efficient partitioning scheme would minimize that loss; moreover the reduction of execution time would be very rewarding.

It is well-known that multiple-way network partitioning problems are NP-hard. Therefore, typical approaches to solving such problems find heuristics that will yield approximate solutions in polynomial time [San89, YC91, She95]. Traditional partitioning problems usually have explicit objective functions; for example, in physical layout it is desirable to have minimal interface signals resulting from partitioning the circuit, and hence, the objective function to be minimized there is the number of nets connecting more than two blocks. The sequential circuit partitioning problem, however, is made harder by the absence of a well-defined objective function; since the ultimate goal is to minimize total area of the circuit, there is no direct physical measure that could serve as an objective function for partitioning. In the remainder of this section, we present a heuristic measure that will be shown to be an effective objective function for the partitioning problem.

To help in the description of the partitioning algorithm, the following terminology is introduced. For a synchronous sequential circuit, such as one shown in Figure 3.12,

An **internal latch** is a latch whose fanin and fanout gates belong to the same combinational block.

A **sequential block** consists of a combinational subcircuit and its associated internal latches.

Boundary latches are latches that act as either a pseudo-primary input or a pseudo-primary output (but not both) to a combinational block, i.e., latches whose fanin and fanout gates belong to different combinational blocks.

A partition of a synchronous sequential circuit $\mathbf{N}$ is the division of the sequential blocks of $\mathbf{N}$ into disjoint groups. A b-way partitioning of the network is described by the b-tuple $(\mathbf{G}_1, \mathbf{G}_2, \ldots \mathbf{G}_b)$ where the $\mathbf{G}_i's$ are disjoint sets of sequential blocks whose union is the entire set of blocks in the network. Each $\mathbf{G}_i$ is said to be a *group* of the partition.

For a given sequential block $\mathbf{B}$, let $L_{\mathbf{B}}$ denote the set of boundary latches incident on $\mathbf{B}$, and for a given boundary latch L, $\mathbf{B}_L$ denotes the set of sequential blocks on the latch L. For each boundary latch L, the *input tightness* τ_{in}, *output tightness* τ_{out}, and the *tightness ratio* τ are defined as

$$\begin{aligned}
\tau_{in}(L) &= \text{maximum combinational delay from any boundary latch to } L \\
&\quad \text{in the unsized circuit,} \\
\tau_{out}(L) &= \text{maximum combinational delay from } L \text{ to any boundary latch} \\
&\quad \text{in the unsized circuit,} \\
\tau(L) &= \begin{cases} \tau_{in}/\tau_{out} & \text{if } \tau_{in} \geq \tau_{out} \\ \tau_{out}/\tau_{in} & \text{if } \tau_{in} < \tau_{out} \end{cases}
\end{aligned} \tag{3.17}$$

where the adjective "unsized" implies that all gates in the subcircuit are at the minimum size. The tightness ratio $\tau(L)$ provides a measure of how advantageous it would be to provide a skew at L.

For each pair of blocks $(\mathbf{B}_i, \mathbf{B}_j)$, define merit μ_{ij} as

$$\mu_{ij} = \sum_{\mathbf{B}_i \overset{L_k}{\leftrightarrow} \mathbf{B}_j} \tau(L_k) \tag{3.18}$$

where $\mathbf{B}_i \overset{L_k}{\leftrightarrow} \mathbf{B}_j$ means that latch L_k lies between $\mathbf{B}_i$ and $\mathbf{B}_j$. μ_{ij} is defined to be 0 if $\mathbf{B}_i$ and $\mathbf{B}_j$ are disjoint. Physically, μ_{ij} is used to measure the figure of merit if $\mathbf{B}_i$ and $\mathbf{B}_j$ are in the same group.

The cost associated with each block, $\mathbf{B}_i$, is c_i, the number of linear programming constraints required for solving $\mathbf{B}_i$. This number can be calculated very efficiently. Assume that group $\mathbf{G}_k$ consists of blocks $\mathbf{B}_{ki}, i = 1, \ldots |\mathbf{G}_k|$. Then the cost of $\mathbf{G}_k$ is defined as $C(\mathbf{G}_k) = \sum_{i=1}^{|\mathbf{G}_k|} c_{ki}$, and the merit of $\mathbf{G}_k$, $M(\mathbf{G}_k) = \sum_{i=1}^{|\mathbf{G}_k|} \sum_{j=i+1}^{|\mathbf{G}_k|} \mu_{ij}$. The problem is now stated as

$$\begin{aligned}
&\text{maximize} && \sum_{k=1}^{N} M(\mathbf{G}_k) \\
&\text{subject to} && C(\mathbf{G}_k) < MaxConstraints.
\end{aligned} \tag{3.19}$$

where N is the number of groups, and $MaxConstraints$ is the maximum number of constraints that one wishes to feed to the linear program. During the partitioning procedure, which proceeds by a sequence of moves between groups, the capacity of each group is allowed to exceed $MaxConstraints$ by a multiplicative factor $\alpha \geq 1$, which is a technique that is used to permit some amount of hill-climbing in finding the optimal solution by permitting the cost of a group to exceed $MaxConstraints$ temporarily.

Now that the partitioning problem has been explicitly defined, a multiple-way synchronous sequential circuit partitioning algorithm based on the algorithm proposed by Sanchis [San89] is applied to generate the subcircuits.

For each group $\mathbf{G}_k$, and each boundary latch L, define the connection number, $\mathbf{\Phi}$, as:

$$\mathbf{\Phi}_{G_k}(L) = |\{\mathbf{B}|\mathbf{B} \in \mathbf{G}_k \text{ and } \mathbf{B} \in \mathbf{B_L}\}|. \tag{3.20}$$

Since each boundary latch connects exactly two blocks, $\mathbf{\Phi}_{G_k}(L) \in \{0, 1, 2\}$. In other words, if $\mathbf{B}_i \overset{L}{\leftrightarrow} \mathbf{B}_j$, then (a) if $\mathbf{B}_i \notin \mathbf{G}_k$ and $\mathbf{B}_j \notin \mathbf{G}_k, \mathbf{\Phi}_{G_k}(L) = 0$, (b) if $\mathbf{B}_i \notin \mathbf{G}_k$ and $\mathbf{B}_j \in \mathbf{G}_k$, or vice versa, $\mathbf{\Phi}_{G_k}(L) = 1$, and (c) if $\mathbf{B}_i \in \mathbf{G}_k$ and $\mathbf{B}_j \in \mathbf{G}_k, \mathbf{\Phi}_{G_k}(L) = 2$.

The *gain* associated with moving $\mathbf{B}$ from $\mathbf{G}_i$ to $\mathbf{G}_j$ is defined as

$$\begin{aligned} \mathbf{\Gamma}_{ij}(\mathbf{B}) \quad = \quad & \sum_l (\tau(L_l)|L_l \in L_{\mathbf{B}} \textit{ and } \mathbf{\Phi}_{G_j}(L_l) = 1) \\ & - \sum_n (\tau(L_n)|L_n \in L_{\mathbf{B}} \textit{ and } \mathbf{\Phi}_{G_i}(L_n) = 2) \end{aligned} \tag{3.21}$$

The first term of (3.21) measures the benefit of moving $\mathbf{B}$ to $\mathbf{G}_j$, while the second measures the penalty of moving $\mathbf{B}$ out of $\mathbf{G}_i$.

Before beginning the partitioning procedure, the number of linear programming constraints, c_i, required for each block i is calculated using the symbolic constraints propagation algorithm. If $c_i \geq MaxConstraints$ for some block $\mathbf{B}_i$, then it is placed in a group alone, and will not be processed later. Let $TotalConstraints = \sum_j (c_j | c_j < MaxConstraints)$. Each remaining block is put into one of the N' groups,

$$N' = \left\lceil \frac{TotalConstraints}{MaxConstraints} \right\rceil, \tag{3.22}$$

such that for each group k, $C(\mathbf{G}_k) < MaxConstraints$. This is an integer knapsack problem, and many heuristic algorithms can be used to obtain an initial partition (see, for example, Chapter 2, [SDK83]). In some cases, it may be impossible to put all blocks into N groups without violating the restriction on $C(\mathbf{G}_k)$ above; if so, the number of groups may be larger than that given in (3.22).

Given the initial partition, the algorithm improves it by iteratively moving one block of partition from one group to another in a series of passes. A block is labeled *free* if it has not been moved during that pass. Each pass in turn consists of a series of iterations during each of which the free block with the largest gain

is moved. During each move, that the number of constraints in a block are not permitted to violate the limit given by (3.19). The gain number, $\mathbf{\Gamma}_{ij}(\mathbf{B})$, is updated constantly as blocks are moved from one group to another. At the end of each pass, the partitions generated during that pass are examined and the one with the maximum objective value, as given by (3.19), is chosen as the starting partition for the next pass. Passes are performed until no improvement of the objective value can be obtained.

After the partitioning procedure has been applied to the circuit, the optimization algorithm described earlier is applied to each group.

The experimental results published in [CSH95] showed considerable improvements when sizing and skew were used in conjunction, as against using sizing alone.

3.5.2 A Convex Programming Formulation

In [SSF95, SSF98], a method for combined sizing and skew optimization is presented for acyclic pipelines. As in the case of the linear programming approach just presented, this procedure utilizes the idea of cycle-borrowing using clock skew optimization to relax the stringency of the timing specification on the critical stages of the pipeline. Instead of the convex piecewise linear delay model in the previous approach, this method uses Elmore delays.

The combined problem of sizing and skew optimization is formulated as:

$$\begin{aligned} & \text{minimize} && \mathit{Area} && \\ \text{subject to } & x_i + \overline{d}(i,j) + T_{setup} && \leq && x_j + P + \delta && (3.23) \\ & x_i + \underline{d}(i,j) && \geq && x_j + T_{hold} + \delta && (3.24) \\ & -X_{max} && \leq && x_i \leq X_{max} && (3.25) \end{aligned}$$

where $\overline{d}(i,j)$ and $\underline{d}(i,j)$ are functions of the gate sizes in the circuit, P is the specified clock period, and X_{max} is the maximum allowable skew magnitude. The *Area* objective function is approximated as the sum of all transistor sizes.

The area of the clocking network is not explicitly included in the formulation for two reasons. Firstly, it is difficult to derive a relation between the skews and the clocking network area. Secondly, the complexity of routing the clock network is such that it is not possible to predict whether a nonzero skew clock tree will necessarily use up more routing resources than a zero-skew tree.

A simple model such as the sum of the skews would not work; consider, for example, the case where FF_1 and FF_2 are two flip-flops that are physically adjacent in the layout of the circuit and have the same skew. Then one may take advantage of their physical proximity by using the same set of buffers to generate the skew for both, as shown in Figure 3.13(a), rather than wastefully adding extra buffers as shown in Figure 3.13(b).

Even so, it is essential to ensure that the expense of clock routing does not run amuck, and therefore, the constraint (3.25) is introduced; the idea is that since the clock tree is likely to consume routing resources when the skew

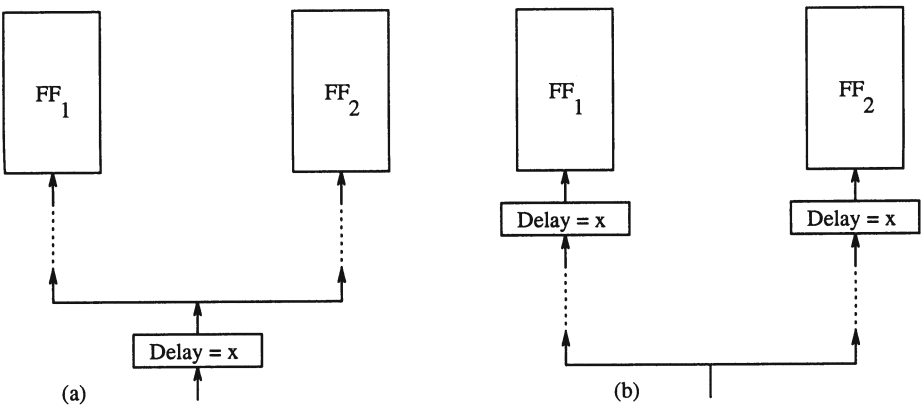


Figure 3.13. Clock network routing considerations.

magnitudes are large, the cost of the clock tree is controlled by limiting the maximum skew magnitude.

Properties of the Optimization Problem. Under the Elmore delay model, it can be shown [FD85] that the gate delays are posynomial functions[3] [Eck80] of the gate sizes. A posynomial function in $\mathbf{y}$ can be transformed into a convex function in $\mathbf{z}$ using the mapping $y_i = e^{z_i}$. Based on this fact, it was pointed out in [Fis90] that the above optimization problem is a *signomial* programming problem [Eck80] and does not, in general, correspond to a convex program. This makes it difficult to arrive at a good solution to the problem.

It can be seen that each long path constraint is of the form

$$\overline{d}(i,j) + x_i - x_j \leq K, \tag{3.26}$$

where $\overline{d}(i,j)$ is the maximum delay between flip-flops i and j (which is some posynomial function of the transistor sizes), x_i and x_j are clock delays to the source and destination flip-flops, and K is a constant. The left-hand side of this inequality is not a posynomial because of the negative coefficient of x_j. If the logarithmic substitution, $x = e^z$, were performed for each clock skew variable x and transistor size variable w in this inequality, the result would not be a convex constraint.

However, performing the substitution $w = e^z$ for each transistor width w appearing in $\overline{d}(i,j)$, while leaving x_i and x_j alone, results is a convex constraint: the left-hand side of the inequality is the sum of $\overline{d}(i,j)$, which is convex in the new z variables, and $x_i - x_j$, which is linear (hence convex), in the variables x_i and x_j[4].

Therefore, it can be concluded that for general sequential circuits, the problem of adjusting both clock skews and transistor sizes to meet long path constraints, while minimizing total gate area, corresponds to a convex optimization program.

Based on these observations, the following approach was proposed to solve this problem in two steps:

1. Neglect the short path constraints (which may be nonconvex [Fis90]) and solve the combined sizing and skew optimization under long path constraints only. This is equivalent to a convex optimization problem and is, therefore, a unimodal problem, i.e., any local minimum is also a global minimum.

2. Resolve any short path constraints that are violated at the end of Step 1 by changing the topology of the circuit and adding buffers in an optimal manner using variation of techniques such as [SBS93], described in Section 2.4.

Apart from easing the solution of the optimization problem, the above strategy has a useful side-effect. Since the presence of short path constraints results in a larger value of the optimal clock period over that achieved in the case where they are neglected, the method above will result in better clock periods than are achievable by keeping the circuit topology the same. It is also hoped that in practical circuits, the number of short path violations, and hence, the number of buffers to be added, will be not be too large.

The approach taken to solve Step 1 is an adaptation of the TILOS algorithm [FD85]. Although more exact methods such as those in [SRVK93] may be used to solve the convex programming problem exactly, these are not practical for use in this situation since the number of variables is too large for the exact algorithm to handle efficiently.

The optimization approach is divided into two stages that are repeated iteratively. In the first, violations of the clocking constraints must be detected. We present a modification of CPM for delay estimation, generalized to handle acyclic sequential circuits. Next, a critical path is defined and identified, and the size of the most sensitive gate on this path is bumped up by a small amount. The iterations continue until all long path-constraints are satisfied for the given clock period.

All of the theory developed so far applies to general sequential circuits. However, from this point, the discussion of this algorithm is specific to acyclic sequential circuits. This restriction is brought about by the fact that the fast timing analysis procedure described below can only handle acyclic circuits (though it may be extended to circuits with cycles).

Detection of Long-Path Constraint Violations. The procedure for timing analysis requires the determination of the optimal skews and the arrival times at all gate outputs. Since the constraint graph for an acyclic pipeline is acyclic, and since timing analysis involves the solution of the longest path problem on this constraint graph, the complexity can be reduced substantially by processing the vertices in topological order [CLR90]. Specifically, the timing analysis procedure is equivalent to applying CPM, described in Section 2.2.2, to the graph, with gates being represented in the normal way, and flip-flops being represented by blocks with "delay" values of $T_{setup} - P - \delta$, where T_{setup}

is the setup time for the flip-flop and P is the applied clock period, and δ is the uncertainty in the clock skew[5].

We point out here that the reference point for the arrival time in each combinational block is the zero skew. For example, an arrival time of a at the output of a flip-flop implies that the clock signal at the flip-flop has been skewed by a. If this flip-flop fans out to a single inverter with a unit delay, then the arrival time at the output of this inverter would be $a + 1$. Since skews may be either positive or negative, it can be seen that arrival times may have either sign.

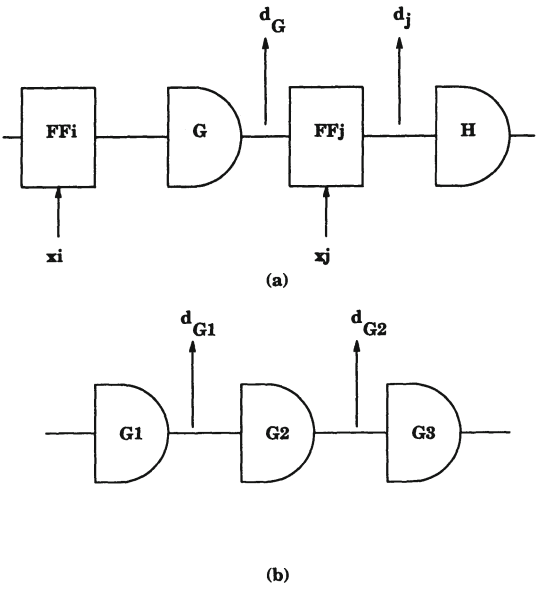


Figure 3.14. Modified CPM.

Consider Figure 3.14(a). We symbolically show the path from FF_i to FF_j as being represented by a single gate with delay $G_{maxdelay}$, which corresponds to the maximum combinational delay between these flip-flops. From (3.23) we have

$$x_i + G_{maxdelay} + T_{setup} \leq x_j + P + \delta \tag{3.27}$$

where x_i is the latest arrival time at the input of G, x_j is the latest arrival time at the input of gate H. Writing $x_i + G_{maxdelay}$ as d_G, the latest arrival time at the input of FF_j, we have the following constraint

$$x_j - d_G \geq T_{setup} - P - \delta \tag{3.28}$$

For a regular combinational gate, as shown in Figure 3.14(b), if d_{G1} and d_{G2} are the latest arrival times at the input of $G2$ and $G3$, then we have the difference constraint

$$d_{G2} - d_{G1} \geq delay(G2) \tag{3.29}$$

From the two difference constraints given above it can be seen that FF_j behaves like a gate with a delay of $T_{setup} - P - \delta$.

It is noteworthy that CPM is valid if the circuit is acyclic; if not, techniques such as those used in [LW83] will have to be applied for analysis. Alternatively, for optimization purposes, a cyclic graph can be forced to be acyclic using the techniques in [BSM94], with additional constraints imposed at points where the cycles are broken.

The method used here for modeling the delay of a gate has been used extensively and successfully in several gate sizing algorithms [SK93]. The delay of a gate can be characterized by its size and the capacitive load that it drives. All transistors of the same type (n or p) within the gate are assumed to have the same size in this algorithm, and the gate delay is calculated by replacing it by an equivalent inverter with parameters W_N and W_P corresponding to the equivalent n and p transistor sizes, respectively.

Each gate is replaced by an RC tree with a driving resistance R_{on} and a capacitive load C_{out}. The Elmore delay of the charging(discharging) network is taken to be the rise(fall) time of the gate. As has been done in previous work on sizing, the on-resistance, R_{on}, is inversely proportional to W_P (W_N) during the charging (discharging) transition. The capacitive load, C_{out}, is composed of two parts: C_{fanout}, the fanout capacitance, and C_{intr}, its intrinsic capacitance. The value of C_{fanout} of gate i is equal to its fanout gate capacitances and is given by

$$C_{fanout}(i) = \beta \cdot \sum_{i=1}^{f} (W_{i,N} + W_{i,P}) \tag{3.30}$$

where $W_{1,N}, W_{1,P}, \cdots W_{f,N}, W_{f,P}$ are the sizes of the n and p transistors in the equivalent inverters corresponding to the f gates that logic gate i fans out to, and β is a constant of proportionality.

The value of C_{intr} of gate i is proportional to its own size and is given by

$$C_{intr} = \alpha \cdot W_i \tag{3.31}$$

where W_i is the size of gate i, given by the sum of the n and p-transistor channel widths, and α is a constant of proportionality.

We can now write the delay of a gate by the following equation [FD85, ME87, SSFD88, SRVK93]:

$$Delay = R_{on} \cdot (C_{fanout} + C_{intr}) \tag{3.32}$$

Unifying Transistor Sizing and Clock Skew Optimization. Initially, all gate sizes are set to the minimum value. The sensitivity-based algorithm performs the following tasks in each iteration:

1. A timing analysis is carried out on the combinational circuit using CPM to identify paths that fail to satisfy the delay constraint. A violation is said to occur if the rise/fall delay at a primary output is ≥ 0, or if the skew at

a flip-flop exceeds the specified maximum skew. In each case, the delay of the path from the primary inputs to the flip-flop or primary output must be reduced. Starting from the flip-flop or primary output with the maximum violation, a critical path is traced back from the primary output with the largest violation, to a primary input.

2. The sensitivity of each gate along the critical path is computed, and the most sensitive gate is identified, so that its sizes may be bumped up. Depending on whether the gate to be sized is undergoing a rise (fall) transition, the p-channel (n-channel) width is bumped up.

Therefore, in each iteration, it is expected that the delay of the critical path will be reduced by a small amount. However, the change in the size of the most sensitive transistor for the current critical path is liable to increase the delays of other paths in the circuit, and hence, as in [FD85], it is important to use a small bumpsize.

The algorithm continues until all timing requirements are met, or until the sensitivity of the most sensitive gate in the most critical path is ≥ 0. At this stage, any further increase in channel widths results only in the increase in delay, and the specified clock period is unachievable. The process of calculating the optimal skews falls out as a natural consequence of this procedure: if the timing requirements are met, then the arrival time at the output of a flip-flop, as calculated by CPM, is the optimal skew to be applied to that flip-flop.

The experimental results published in [SSF98] showed considerable improvements when sizing and skew were used in conjunction, as against using sizing alone.

3.5.3 A Procedure that Incorporates the Cost of the Clock Tree

Both the linear programming and convex programming technique described here control the cost of the clocking network only by placing bounds on the clock skews. An alternative procedure in [XD97] optimizes the cost of the clock tree directly, but uses more approximate models for gate sizing. The optimization problem is set up as

$$\text{minimize } C(T, \mathbf{x}) = \lambda L(T) + \gamma \cdot \Phi(\mathbf{x}) \tag{3.33}$$

where $C(T, X)$ is a cost function that is dependent on the routing topology T of the clock tree and on the vector of gate sizes $\mathbf{x}$. The functions $L(T)$ and $\Phi(\mathbf{x})$ represent the total wire length of the clock tree, and the power dissipation of the sized gates, respectively, and λ, γ are appropriately chosen weighting parameters. The cost function is optimized using simulated annealing [KGV83].

The algorithm first derives bounds on the relative skew at various sinks. If a pair of latches i and j is connected by combinational logic, then it is possible to derive upper and lower bounds on the value of the difference in skew $x_i - x_j$ using expression (3.9); these are referred to as the positive skew bound, PSB_{ij}, and the negative skew bound, NSB_{ij}. For nonadjacent sinks, no such bound

exists. Note that since the expression (3.9) is dependent on the delays, the bounds are derived over all circuit delays that correspond to allowable gate sizes.

For a complete understanding of this procedure, it is necessary to understand the DME procedure [KR95] fully (see Section 3.3.3). We will outline the essentials of the method to provide a general overall understanding.

Unlike the case of a Euclidean geometry, under a Manhattan geometry, there are several paths between two points that have the same distance. Therefore, when the tapping point (see Section 3.3) is chosen during a bottom-up zero-skew merge, it may lie at a distance $x \times l$ away from one node along *any* of these routes. It can be shown that the locus of possible tapping points forms a line segment, called a *merging segment.*

In the problem at hand, since the skew is not fixed, but bounded, this permits a variation in the location of the tapping point, and therefore, a variation in the location of the merging segment of the DME procedure. A move in simulated annealing corresponds to changing the position of a merging segment in such a way that the skew bounds are satisfied.

The gate sizing results are stored in the form of a look-up table. The minimum power of each combinational block for various sets of skews is determined and stored in this table. When the clock network is perturbed, the skews are mapped on to the closest set of skew values listed in the table, and the table is accessed to determine the corresponding power dissipation. The cost of this configuration is evaluated and fed into a simulated annealing loop that finds the optimal solution to the problem.

The tradeoff in this approach is that it provides accuracy in modeling the cost of the clock tree network, and an approximate and coarse modeling of the sizing procedure; the reverse was true of the previous two methods.

3.6 WAVE PIPELINING ISSUES

The treatment in [GLC94] provides an excellent illustration of the idea of wave pipelining, and is summarized here. A conventional combinational circuit and its associated timing are shown in Figure 3.15(a). The data flows out of the input registers on the onset of the clock, and propagates through the logic, reaching the output registers in time for the onset of the next clock.

In contrast, in a wave pipelined system, the propagation time for the data through the logic may be larger than the clock period. As an example, in Figure 3.15(b), the clocks at the input and output register are skewed. The data that leaves the input registers reaches the output registers after more than one clock period. During this time, the propagation of the second set of data from the input registers has commenced at time T_{clk}. Therefore, there is a period of time when two different signals corresponding to two different clock periods are flowing through the combinational logic, corresponding to two "waves." To ensure correct operation of the circuit, it is imperative to ensure that the two waves do not collide. The advantage of wave pipelining over conventional pipelining is that it may not require the use of as many registers,

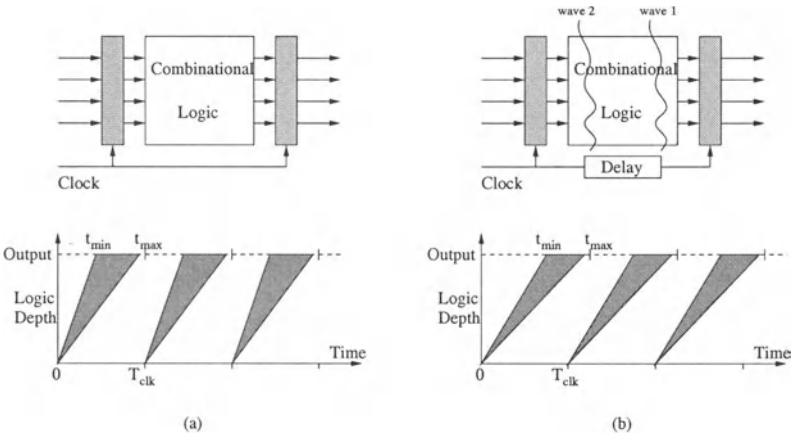


Figure 3.15. Illustrating the idea of wave pipelining.

leading to potential savings in hardware and in the overhead of setup and hold times at each register. Moreover, the overhead of clock distribution is also reduced when the number of registers is reduced.

The use of skews usually implies that more than one wave moves through at least one of the combinational segments during a clock period. As an example, in Figure 3.2, when the first data set reaches the output of gate G2 after two time units (i.e., one clock period), the second data set is sent into the circuit at the input to gate G1. Since there is only one combinational path between IN and F1, there is no question of data collision, but in general, this may not be true.

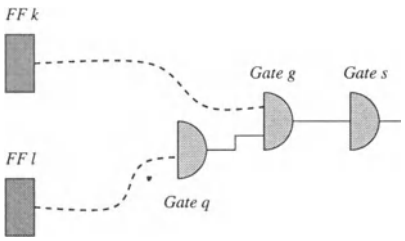


Figure 3.16. An example illustrating logic signal separation constraints.

The work in [JC93] proposed an alteration of the clock skew formulation of Fishburn [Fis90] to incorporate constraints that enforce the requirement that no two waves collide during transmission; these constraints are referred to as logic signal separation constraints. Figure 3.16 shows two flip-flops, k and l, and a set of combinational logic gates, q, g and s. Node q is any predecessor gate of g such that flip-flop l lies in its fanin cone, and gate s is any successor of node g. Let the skews at the flip-flops k and l be x_k and x_l, respectively. The logic signal separation constraint under a clock period P is stated mathematically

as follows:

$$x_k + \text{delay}(k, s) \leq P + x_l + \text{delay}(l, q) \tag{3.34}$$

The left hand side of this equation represents the arrival time of the signal from the current wave at the output of the successor of g. This arrival implies that the value at g is no longer important for the current wave. The right hand side is the arrival time at the output of q during the next wave. Simply stated, the inequality says that it is essential for the inputs of g to be stable for as long as its output influences the current wave. In case g has multiple predecessors and successors, the relationship (3.34) must hold between each predecessor-successor pair.

Earlier in this chapter, several techniques for combined sizing and skew were described. None of them explicitly considers the logic signal separation constraints in the form in which they were published. However, it is relatively easy to incorporate them into the formulations. For the linear programming formulation, the output arrival time at each gate is available as a variable, and it is an easy matter to add constraints of the form (3.34) at $m_s \leq p_q + P$. For the convex programming formulation, this constraint can be checked during the application of CPM, and any violation can be flagged as a constraint violation to be reconciled by altering the sizing and/or skew solution.

3.7 DELIBERATE SKEWS FOR PEAK CURRENT REDUCTION

In synchronous circuits, all of the flip-flops switch at the latching edge of the clock. This simultaneous switching of flip-flops requires a large switching current, which in turn creates a significant voltage drop on the power distribution network, creating the need for multiple pins to distribute the power supply. In [VHBM96] deliberate clock skews are used to reduce the dynamic transient current drawn from the supply pins. This reduction in the transient current reduces the number of supply pins required by the chip, and therefore, its packaging cost.

The objective of introducing skews at the flip-flops is to ensure that all of them do not switch at the same time and hence, do not draw current at the same time; this serves to reduce the peak current. The problem is modeled as an integer linear program that minimizes the peak current. Each flip-flop may have one of n possible skew values. Correspondingly, for each flip-flop a set of n binary variables is used, of which exactly one variable is permitted to be 1, and the skew of the flip-flop corresponds to this variable. The clock period of the circuit imposes constraints on the possible skew value for each flip-flop, these timing constraints are incorporated in the linear program. Experimental results presented in [VHBM96] show a significant reduction in the ground bounce using this technique. However, the use of an integer linear programming formulation restricts the technique to system level design with few tens of modules. It is possible that the application of heuristics to this technique may permit it to be extended to larger designs, at the cost of sacrificing optimality.

A method for heuristic minimization of peak current by using clock skew is presented in [VBBD96, BVBD97]. Experimental results on circuits with up to

550 flip-flops indicate an average reduction of about 30% in the peak current. This method only minimizes the current peak directly caused by clock edges using a genetic algorithm. The current waveform is approximated as a triangle to allow efficient calculation of the total peak current in the circuit. The genetic algorithm based solution results in a required skew at each flip-flop. Since clock skew control for individual flip-flops is difficult, the flip-flops are clustered into a user specified number of clusters, and each flip-flop in a cluster has the same skew. A heuristic is used to attempt to cluster the flip-flops in such a way that there is a minimal loss in optimality due to this simplification.

3.8 CONCLUSION

The utility of deliberate skews for optimizing the performance of VLSI circuits has been demonstrated, and algorithms for performing skew scheduling and period minimization have been presented. Deliberate skews can also be used in conjunction with other timing optimization strategies and for minimizing the peak current in the supply network.

Until recently, there has been a great reluctance to alter the clock network and attempt a nonzero-skew solution. However, recently, an increasing number of designers have been willing to utilize skews for performance enhancement. Small amounts of skews can easily be provided by making minor changes in the sizes of the final buffers in the clock tree that feed the clock sinks, and altering skews in this manner is a relatively painless manner in which the optimization could be applied. For larger skew magnitudes, a more careful design of the clock network is essential; for high-performance applications, the gains outweigh the costs of this effort.

Notes

1. The reader is cautioned that the area-delay curves for Stg_1 and Stg_2 are, in reality, distinct from each other. To reduce the amount of clutter in the figure, in this example, we have shown all points on the same curve.

2. In this example, the change in the cost of the clock network has been neglected. It should be noted that it is easy to provide small skews by making minor changes in the sizes of the buffers driving the leaf-level nodes, and it possible to do this cheaply for small skew magnitudes.

3. A posynomial is a function g of a positive variable $\mathbf{w} \in \mathbf{R}^n$ that has the form $g(\mathbf{w}) = \sum_j \gamma_j \prod_{i=1}^{n} w_i^{\alpha_{ij}}$ where the exponents $\alpha_{ij} \in \mathbf{R}$ and the coefficients $\gamma_j > 0$. Roughly speaking, a posynomial is a function that is similar to a polynomial, except that (a) the coefficients γ_j must be positive. (b) an exponent α_{ij} could be any real number, and not necessarily a positive integer, unlike the case of polynomials.

4. This concept can be generalized to any optimization problem that can be divided into two separate classes $w_1, \cdots, w_n$, and $x_1, \cdots, x_m$, with each constraint is of the form

$$P(w_1, ..., w_n) + C(x_1, ..., x_m) \leq K, \tag{3.35}$$

where P is a posynomial function, C is a convex function, and K is a constant. Such a problem can be transformed into a convex program by performing the substitution $w_i = e^{z_i}$ while leaving the x_i variables alone.

5. Note that the word "delay," when used in reference to flip-flops, is applied in a loose sense here. The "delay" of a flip-flop as defined here is not the propagation delay of the gates within the flip-flop, but is a mathematical tool that can be used to apply CPM to a sequential circuit to check for delay violations in the presence of clock skews.